

SESSION 12

Technical Architecture Design, Solutioning & Architecture Review

From agreed requirements to a solution you can defend — with ATAM as the discipline underneath.

Today

01

Mining the Backlog — Forces & ASRs

02

Use Case Modeling — Behavior & a First Candidate

03

Quality Attributes, Scenarios & the Utility Tree

04

Styles, DDD & Microservices — and the Cloud

05

Documenting the Solution

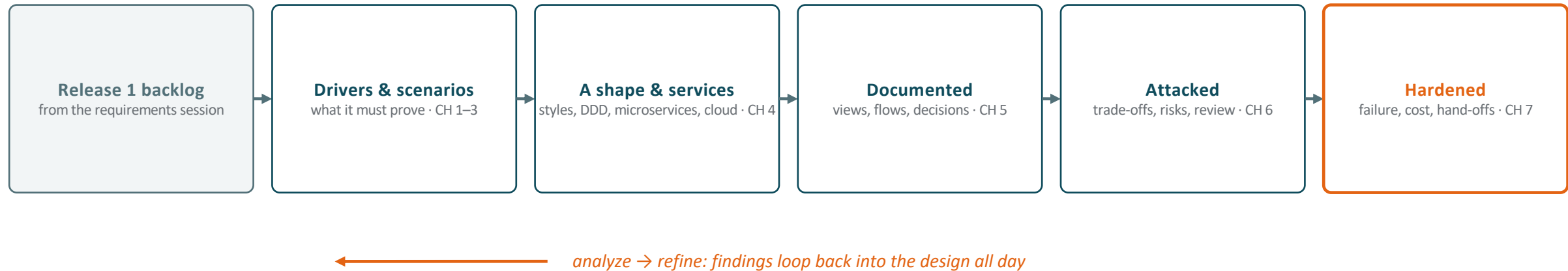
06

Analyzing & Reviewing — the ATAM Discipline

07

Hardening, Hand-offs, Cost & Effort

From your backlog to a defended design



Working rule for the course: everything you produce is a candidate — it exists to be tested against scenarios, challenged in review, and revised without ego. That loop, not any diagram, is the skill.

01

Mining the Backlog — Forces & ASRs

You arrive holding a baselined Release 1 backlog. This chapter mines it: load-bearing stories, the register rows with structural pull, the parked quality words — filtered into your ASR shortlist, the forces every later decision answers to.

THE COURSE

- 1 Mining the Backlog — Forces & ASRs**
- 2 Use Case Modeling — Behavior & a First Candidate
- 3 Quality Attributes, Scenarios & the Utility Tree
- 4 Styles, DDD & Microservices — and the Cloud
- 5 Documenting the Solution
- 6 Analyzing & Reviewing — the ATAM Discipline
- 7 Hardening, Hand-offs, Cost & Effort

You baselined Release 1. Can they start building?



“What runs where?”

One system or several? What is deployed, on what, in which region?



“What talks to what — and how?”

Which calls are synchronous? What happens in what order when someone places an order?



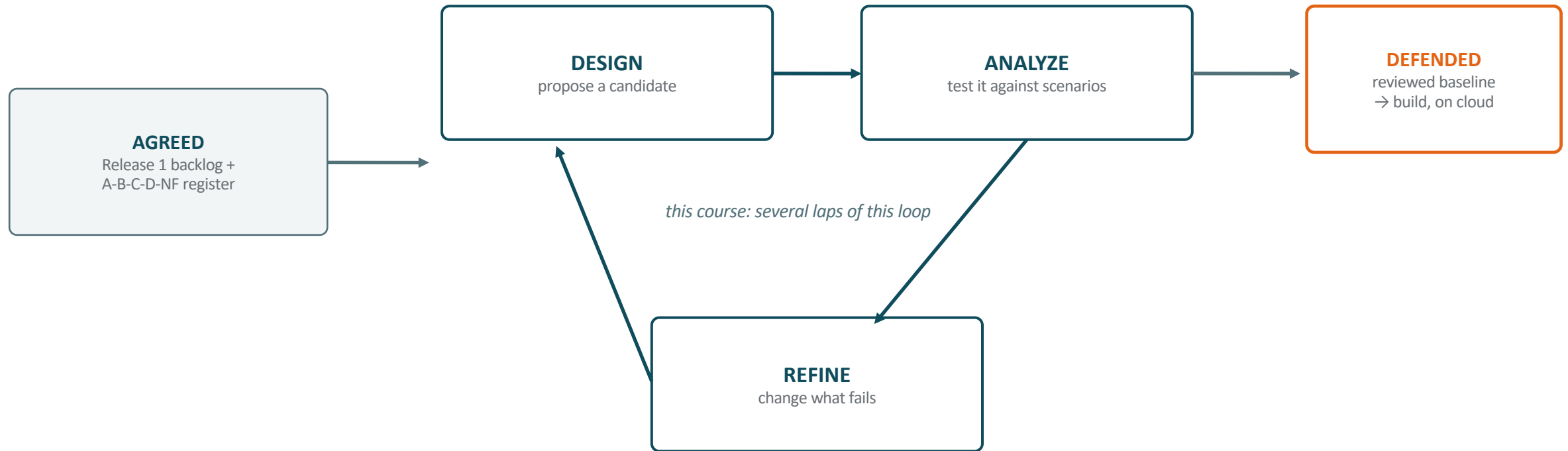
“A third-party dependency dies mid-transaction. Then what?”

The dependency sits on your register — but the backlog has no answer for failure.

THE RULE A backlog says **WHAT** to build and in what **ORDER**. Architecture says **HOW** it hangs together — and what that costs.

Engineering’s questions on day one — all architectural, none in the backlog.

Requirements are agreed. Now: design → analyze → refine



“Candidate” means falsifiable. A candidate solution is a claim you can test: “this shape delivers those scenarios at that cost.” If nothing could prove it wrong, it isn’t a design — it’s a wish.

Turn a list into design material — four artifacts

A backlog says **WHAT** to build, in what **ORDER**. To design, you need four things it doesn't contain — Chapters 1 and 2 build them, from the backlog itself.

1

The ASR shortlist (this chapter)
the FORCES on structure

feeds: becomes v0's proof list in Chapter 2 — and the judge for every Chapter 4 decision

2

Use cases, with failure paths (Chapter 2)
the BEHAVIOR, end to end

feeds: mined for events in Chapter 4 · the critical flow becomes Chapter 5's sequence · the failure paths are what breakers & sagas answer

3

The domain sketch (Chapter 2)
the VOCABULARY — named nouns

feeds: makes “one owner per fact” possible: Chapter 4 assigns these nouns to services

4

Candidate v0 + proof list (Chapter 2)
a starting point you may ATTACK

feeds: the rest of the course refines exactly this: measured (CH 3), shaped (CH 4), documented (CH 5), attacked (CH 6), hardened (CH 7)

THE RULE Nothing in these chapters is an exercise for its own sake — every artifact is consumed by a named later step.

What the backlog gives you — and what it can't

A GOOD STORY — AS BASELINED

US-7 · Pay with a saved card

As a returning customer, I want to pay with my saved card, so that checkout takes seconds instead of minutes.

AC1 Given a card on file and a passed eligibility check, when I confirm the purchase, then I receive an order confirmation with price and quantity.

AC2 Given the purchase completes, then the transaction is recorded and retrievable.

WHAT IT CANNOT TELL YOU

- System shape — one deployable thing or many? What are the pieces?
- Quality — how fast, how available, how secure, at what cost? (“fast” got parked as a non-functional)
- Data ownership — who is the source of truth for orders? for product data?
- Failure — what happens when the payment provider is down mid-purchase?

Same project, two instruments:

The backlog orders the work by value and risk. The architecture shapes the system that all those stories run inside. You need both — and today you already own one of them.

THE RULE Don't redo requirements. Mine them: the backlog is the raw material of architecture.

Find the load-bearing stories



Volume
everyone hits it, all day



Money moves
payments, trades, refunds



Regulation
auditable, mandated, reviewed



Integration
leaves your system's hands



Unknowns
nobody has done it here before

Worked scan — a typical commerce backlog:

LOAD-BEARING

“Search the catalog” — Volume + a data problem — everyone hits it, all day, on every device

LOAD-BEARING

“Check out and pay” — Money + integration + regulation, all at once — the hardest story in most backlogs

LOAD-BEARING

“Export order history for finance” — Regulation, and cross-cutting — every other story feeds it

PARTITION

“Filter by category” — A partition wall: real value, no structural pull — build it anywhere

THE RULE Architecture effort follows load-bearing stories — not backlog rank order.

The quality words you parked are still unresolved

“Search feels fast”

parked NF item

Fast for whom? Measured where? Under what load? At peak?

“Customer data is secure”

parked NF item

Against which threats? Proven how? Logged and alerted in what time?

“Available in business hours”

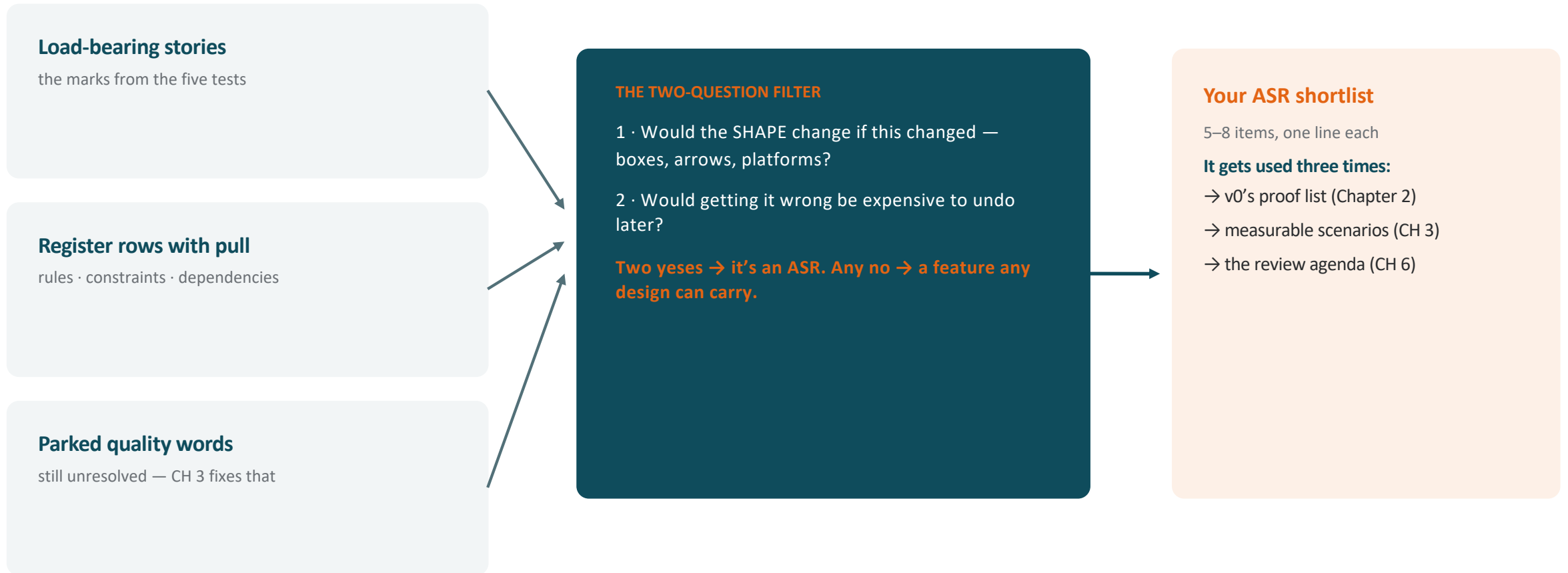
parked NF item

Which functions? Degraded how when a third-party feed fails?

You did the right thing parking them. Vague quality words don’t belong in stories. But parked ≠ resolved — and “we’ll tune it later” is how systems fail their first peak day.

THE RULE Every parked quality word becomes a measurable scenario in Chapter 3 — that is where they get resolved.

From marks to ASRs — three sources, two questions



THE RULE Marks in, shortlist out — and the shortlist is what the rest of the course runs on. Nothing you marked is decorative.

Architecturally significant requirements (ASRs)

ASR = a requirement that shapes structure: change it, and boxes, arrows or platforms move. Stories can be ASRs — but so can rules, constraints and dependencies.

A worked shortlist — five ASRs, only one of them a story:

STORY

Pay online, no staff in the loop — Forces real-time pricing, payment execution, settlement — the transactional core.

RULE

Eligibility check before purchase — Forces a compliance checkpoint INSIDE the checkout flow — order matters.

RULE

Auditable record of every transaction — Forces an immutable, queryable record trail no component can bypass.

DEP

Third-party price/stock feed, SLA unconfirmed — Forces a failure story: cache? degrade? suspend selling? Someone must decide.

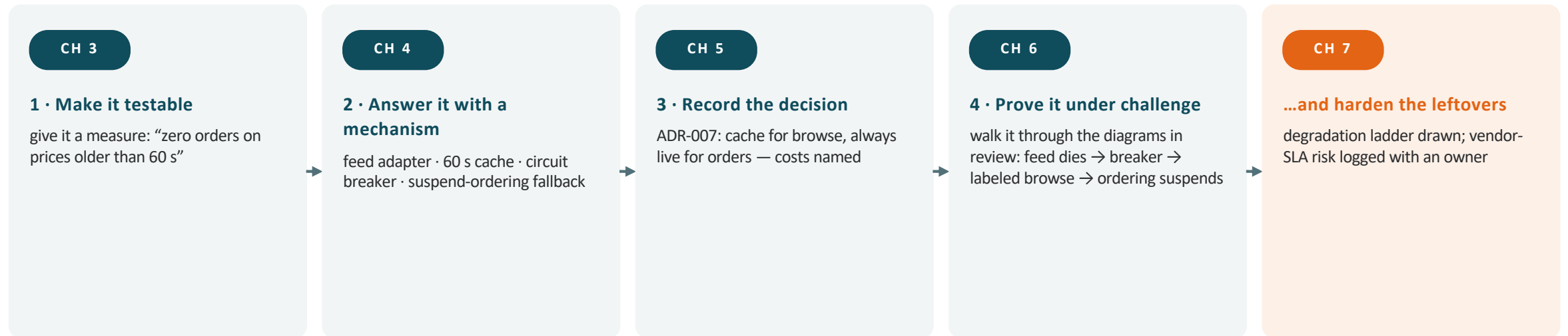
CONSTR

Phone and desktop — Forces one backend serving two clients — API-first, not page-first.

THE RULE List your ASRs before you draw anything — they are the forces your candidate must answer.

What you do with an ASR — follow one through the course

THE ASR “Third-party price/stock feed, SLA unconfirmed — and no order may ever use a stale price.”



Today, in the lab: you only need step 0 — the shortlist itself. The course does the other four to it, chapter by chapter, starting with Chapter 2’s proof list.

THE RULE An ASR no mechanism answers is a hole. A mechanism no ASR demands is gold plating. Reviews hunt for both.

What leaves this chapter with you

- ✓ The marks: load-bearing stories, drivers, constraints, parked words
- ✓ The two-question filter — shape? expensive to undo?
- ✓ Your ASR shortlist — the forces (artifact 1 of the morning's four)
- ✓ Where the shortlist goes: proof list (CH 2), scenarios (CH 3), review agenda (CH 6)

NEXT · CHAPTER 2

You know the forces. Nobody has written the behavior.

Use case modeling — properly: actors, one-page diagrams, fully-dressed flows, and the failure paths where architecture lives.

02

Use Case Modeling — Behavior & a First Candidate

The backlog delivered the pieces; nobody wrote what happens end to end. This chapter is the method for reconstructing it: actors, the use case diagram, one fully-dressed description with its failure paths — then behavior becomes a domain sketch and your first candidate solution.

THE COURSE

1 Mining the Backlog — Forces & ASRs

2 Use Case Modeling — Behavior & a First Candidate

3 Quality Attributes, Scenarios & the Utility Tree

4 Styles, DDD & Microservices — and the Cloud

5 Documenting the Solution

6 Analyzing & Reviewing — the ATAM Discipline

7 Hardening, Hand-offs, Cost & Effort

Ask your backlog: “what happens when a customer buys?”

WHAT STORIES GIVE YOU

Units of DELIVERY.

Small, independent, testable slices, ordered by value and risk — deliberately. Perfect for planning sprints; each answers “what do we build next?”

Ask them “what happens, start to finish?” — every story raises its hand, none answers.

WHAT DESIGN NEEDS

Units of BEHAVIOR.

One customer goal, end to end, in order — with the failure paths on the page. That is what boundaries, sequences and failure design are made from.

This chapter’s method reconstructs it from the slices: use case modeling.

“Shouldn’t this already exist?” — sometimes, yes.

Document-driven shops hand you use cases from analysis. A story-based requirements process deliberately never writes the end-to-end view — so reconstructing it is design’s first move, not a requirements failure. Consolidation, not re-elicitation: nobody is redoing requirements today.

THE RULE Stories stay the units of delivery. Use cases become the units of system behavior — you need both, for different jobs.

Actors: everyone and everything with a goal, outside the boundary

Actor	Type	Their goal / role	Where they came from
Shopper	Primary — human	buy the right product quickly	the “As a...” clause of most stories
Back-office staff	Primary — human	resolve issues, oversee orders	support & ops stories
Payment provider	Supporting — system	authorize and settle payment	the dependency register
Stock & price feed	Supporting — system	supply current price and availability	the dependency register

Test 1 — outside the boundary?

Your database is INSIDE the system — not an actor. The payment provider is outside — an actor. If you build and operate it, it’s in the box.

Test 2 — one goal per actor name?

“Admin” is usually three actors wearing one badge — support agent, compliance reviewer, catalog manager. Split by GOAL, not by job title.

THE RULE Humans from the “As a...” clause, systems from the dependency register — an actor census takes five minutes and prevents an hour of confusion.

The use case diagram: goals on one page

Stories → use case (consolidation)

US-7 Pay by card · US-8 Confirmation · US-9 Eligibility check · US-10 Terms acknowledged
...are four slices of ONE customer goal:

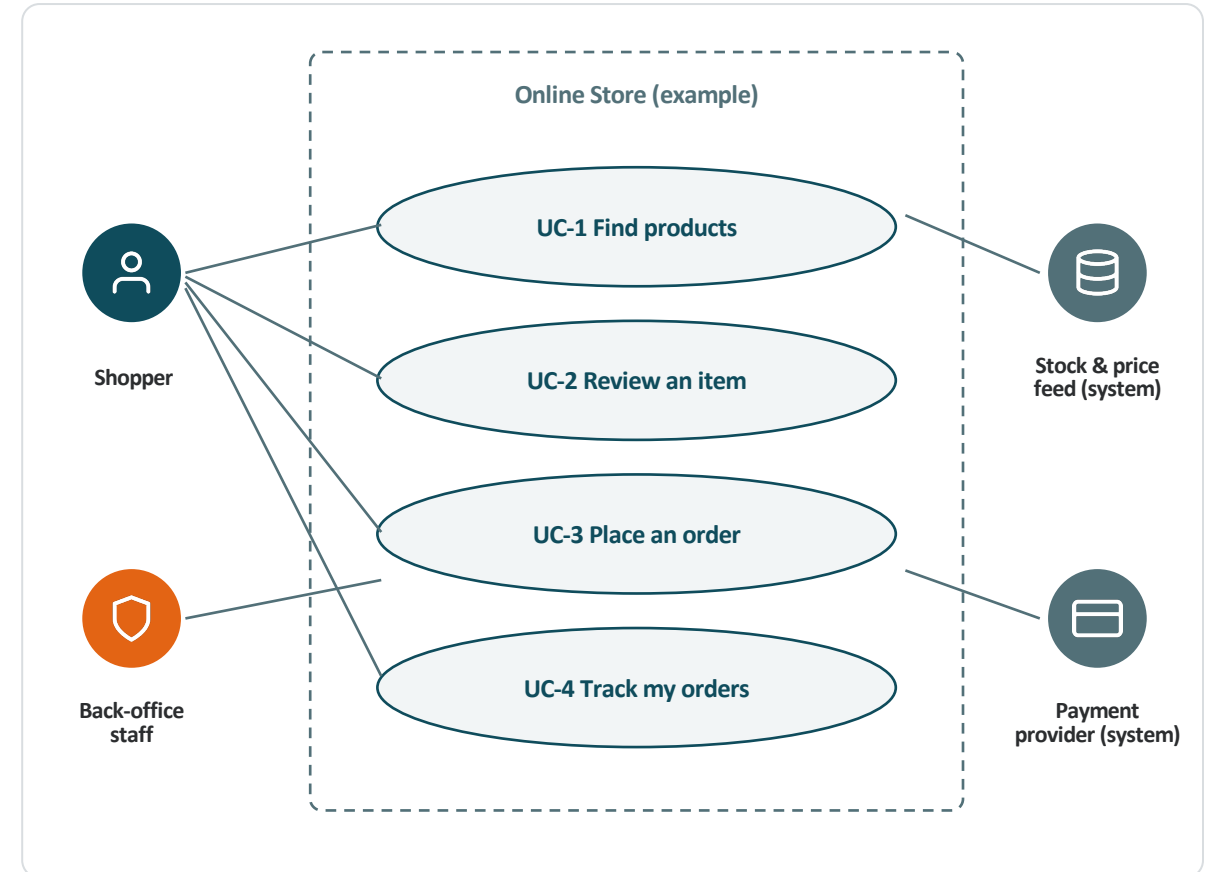
UC-3 Place an order — one flow, with its failure paths.

Reading the picture

Stick figures/icons = actors · ellipses = goals · the box = your system's boundary · lines = "takes part in".

How many is healthy?

3–7 use cases for a system slice. Thirty means you listed features, not goals — consolidation went missing.



THE RULE Many stories → one goal → one ellipse. The diagram is a table of contents for behavior — the descriptions are the book.

Anatomy of a use case — and how dressed it needs to be

The template (fully dressed)

Name + goal — a verb phrase the primary actor would say

Primary actor · supporting actors

Preconditions — what must already be true

Trigger — the event that starts it

Main flow — numbered; the actor-acts / system-answers rhythm; intent, never UI (“confirms the order”, not “clicks the teal button”)

Extensions — numbered off the step they fork from (2a, 5a...): the failure and alternate paths

Postconditions — what is true on success (and on failure)

Three dress levels (Cockburn)

Brief — one paragraph: goal + outcome. For most use cases.

Casual — a few informal paragraphs with the main variations.

Fully dressed — the whole template, failure paths included.

The economics: fully-dress ONLY the load-bearing 3–5 — your ASR shortlist from Chapter 1 tells you which. Brief the rest. Dressing all thirty is how use case modeling got its bad reputation.

THE RULE Write intent, not interface — “confirms the order” survives every redesign; “clicks the teal button” dies with the mock-up.

UC-3 · Place an order

Actor: shopper · **Goal:** the item is ordered · **Trigger:** taps Buy

Preconditions: signed in · an item is selected

Main flow

- 1 Customer taps Buy on the item page
- 2 System runs the eligibility & fraud check (business rule)
- 3 System shows the terms; customer acknowledges (business rule)
- 4 Customer sets quantity and confirms
- 5 System fetches current price & stock — the supplier feed (dependency)
- 6 System places the order for payment & fulfilment (dependency)
- 7 System confirms to the customer and writes the audit record (constraint)

Extensions (failure paths)

2a Eligibility fails → explain why, stop the purchase, route to support. No override.

5a Feed unavailable → suspend ordering with a clear message; browsing continues. Never sell on a stale price.

7a Record write fails → the order must not silently succeed. (Hold that thought for Chapter 4.)

Why this beats four stories for design

One end-to-end goal · explicit ordering (eligibility BEFORE payment) · failure paths on the page — everything Chapter 4 will need.

Extensions are where the architecture lives

2a — the eligibility check fails

a compliance gate must sit INSIDE the flow, before money moves

boundary & sequencing — Chapters 4–5

5a — the supplier feed is unavailable

a failure policy: timeout, breaker, fallback, suspend-vs-degrade

resilience design — Chapter 4

7a — the record write fails

the order and its audit trail must succeed or fail together

saga & buffering — Chapters 4, 7

The test of a finished use case: at least two failure extensions, each one answerable with a design decision. No extensions = a brochure, not a use case.

THE RULE Happy paths sell the feature. Extensions design the system.

How use case modeling goes wrong — and the fix

UI scripting

“1. User clicks Buy Now 2. System shows modal 3. User clicks teal Confirm...”

Write INTENT: “customer confirms the order”. Flows must survive a redesign of every screen.

CRUD explosion

Create Product · Read Product · Update Product · Delete Product... ×10 tables

Use cases are actor GOALS, not table operations. “Maintain the catalog” — one case, briefly dressed.

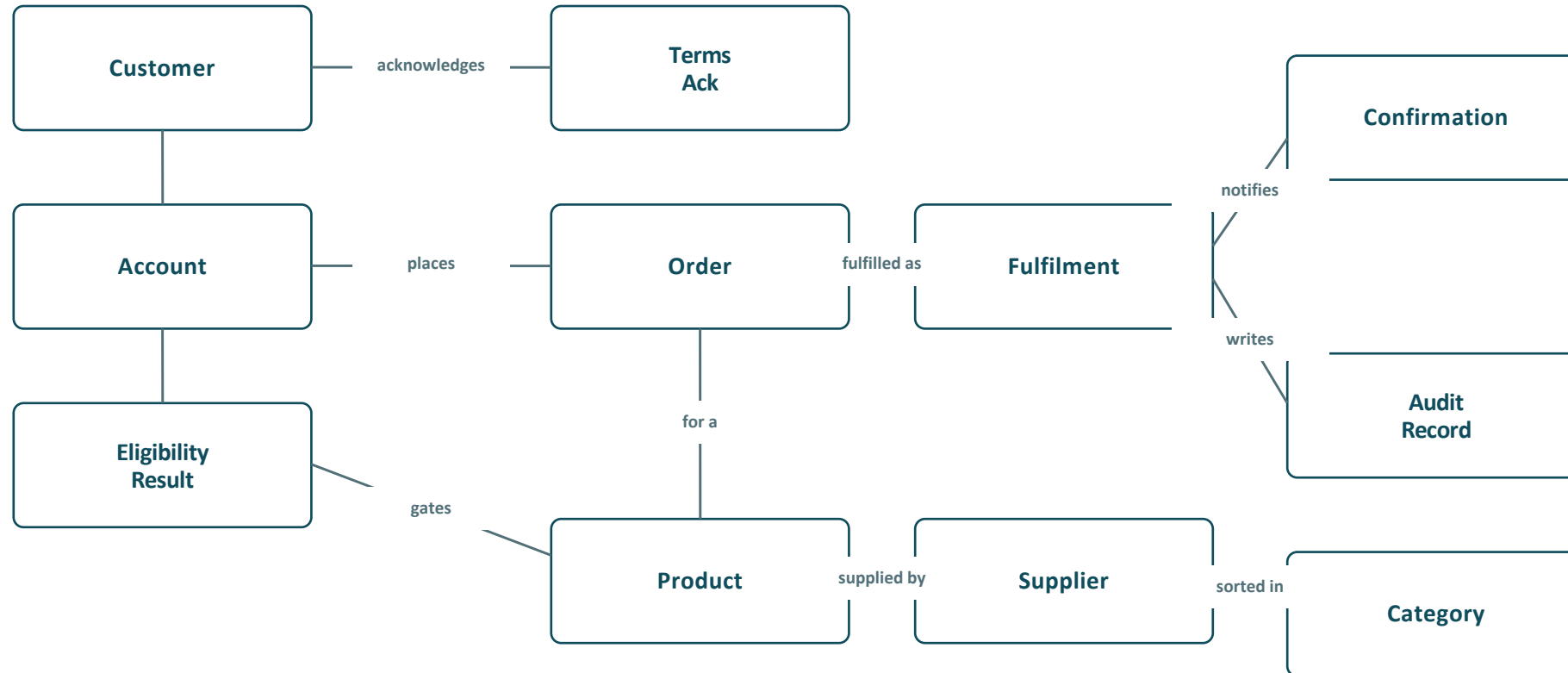
include / extend ceremony

arrows everywhere: «include» «extend» generalization webs nobody can read

Skip the relationships today. Numbered flows + extensions carry everything a designer needs.

THE RULE Goals, not screens. Goals, not tables. Flows, not notation ceremony.

Sketch the domain — ten nouns, one whiteboard



Read it aloud

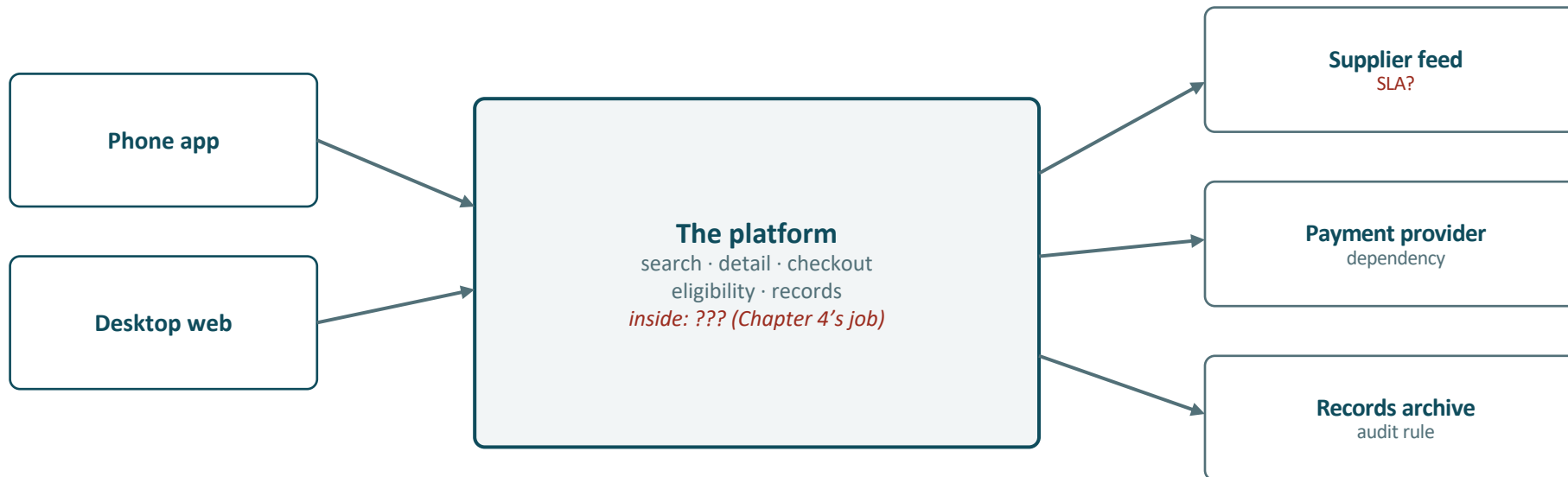
“A customer places an order for a product; it is fulfilled; the fulfilment is confirmed and audit-recorded.”

If the sentence is clumsy, the model is wrong.

NOT a database design — a shared vocabulary. Tables come much later, per service.

THE RULE Order ≠ Fulfilment, Product ≠ Stock. Naming the nouns now is what makes “who owns this data?” answerable in Chapter 4.

Candidate v0 — honest boxes, hard questions



What v0 must prove

- Search stays quick at peak
- No order on a stale price
- Every order leaves an audit trail
- One backend, two clients
- Survives a feed outage

THE RULE The proof list IS your ASR shortlist, rewritten as promises — v0's boxes are sketch, the list is substance.

Model the behavior of your case

Teams of 4–5 · inputs: your case backlog, your register, your Chapter 1 ASR shortlist · flip-chart or A3 per team

1

Take an actor census

Humans from the “As a...” clauses, systems from your dependency register. Type each (primary / supporting), one goal per actor — split any “admin”.

2

Consolidate stories into use cases

Group your backlog into 3–7 use cases — actor goals, not features. Name each as a verb phrase the actor would say. Draw the one-page diagram: actors, ellipses, boundary.

3

Pick the load-bearing ones

Against your ASR shortlist: which 2–3 use cases carry the architecture? Mark them — only those get fully dressed.

Worked template — step 2, one row done for you: *“US-7 + US-8 + US-9 + US-10 → UC-3 Place an order (primary actor: customer).”*

Model the behavior — continued

4

Fully dress ONE use case

Your most load-bearing goal, in the full template: numbered main flow (intent, not UI), at least TWO failure extensions, postconditions. Follow the UC-3 worked example.

5

Sketch the domain

8–12 nouns from your flows and their links, whiteboard style. Read it aloud as sentences — clumsy sentence, wrong model.

6

Draw candidate v0 + its proof list

Clients, your platform (question marks welcome inside), external systems. Then the important half: your ASR shortlist rewritten as 4–5 “what this must prove” lines.

Model answers on the next slide — no peeking until your v0 is on paper.

Model answers — compare, don't copy

Actors & use cases (steps 1–3)

- Census: customer, back-office staff + payment provider, stock feed (system)
- UC-1 find · UC-2 review · UC-3 place an order · UC-4 track orders
- Load-bearing: UC-3 (money+integration+regulation), UC-1 (volume)

The dressed use case & domain (steps 4–5)

- UC-3 written out — as on the worked slide, with extensions 2a / 5a / 7a
- Two failure extensions minimum — each answerable by a design decision
- Domain: Customer, Account, Order, Fulfilment, Product, Supplier, Category, Confirmation, Audit Record, Eligibility Result

Candidate v0 (step 6)

- Two clients → one platform (contents: honest question marks)
- Three externals: supplier feed (SLA!), payments, records archive
- Proof list = the ASR shortlist as promises: quick search at peak · no stale-price orders · audit trail always · one backend, two clients · survives a feed outage

THE RULE Different consolidations with good reasons beat identical ones with none — the reasoning is the deliverable.

What leaves this chapter with you

- ✓ An actor census — every goal-holder, human and system
- ✓ 3–7 use cases, one fully dressed with failure extensions (mined in Chapter 4, sequenced in Chapter 5)
- ✓ A domain vocabulary (becomes data ownership per service in Chapter 4)
- ✓ Candidate v0 + proof list (the rest of the course refines exactly this)

NEXT · CHAPTER 3

Your proof list says “quick”, “survives”, “always”.

Review committees don’t accept adjectives. Time to make every quality claim measurable.

03

Quality Attributes, Scenarios & the Utility Tree

The backlog left three vague quality words parked: “fast”, “secure”, “available”. This chapter resolves them — five quality attributes, a six-part scenario form that makes each one measurable, and a utility tree that tells you where review effort goes.

THE COURSE

- 1 Mining the Backlog — Forces & ASRs
- 2 Use Case Modeling — Behavior & a First Candidate
- 3 Quality Attributes, Scenarios & the Utility Tree**
- 4 Styles, DDD & Microservices — and the Cloud
- 5 Documenting the Solution
- 6 Analyzing & Reviewing — the ATAM Discipline
- 7 Hardening, Hand-offs, Cost & Effort

Quality attributes — and the tensions between them



Performance

results while the customer still cares



Availability

peak hours are the only hours that count



Security

other people's money and identities



Cost

no budget is infinite — the bill must be visible



Maintainability

new features without open-heart surgery

The tensions — you cannot maximize all five:

Availability ↔ Cost

Two regions double the calm — and much of the bill.

Security ↔ Performance

Compliance checks sit INSIDE the checkout path.

Performance ↔ Cost

Caching prices is fast and cheap — until an order uses a stale one.

Your five, by name: performance, availability, security, cost, maintainability. Others exist (usability, portability...) — these five are the ones reviewers ask about, and cost is a first-class member, not an afterthought.

THE RULE Architecture is the art of picking which quality you'll pay for — silence just means the system picks for you.

From adjective to measurement

WEAK

v1 — the parked words

“Search feels fast.”

Untestable. Whose feeling? Which search?

CLOSER

v2 — a bare number

“Search responds in under 2 seconds.”

Better — but under what load? measured where? and what happens beyond it?

REVIEW-READY

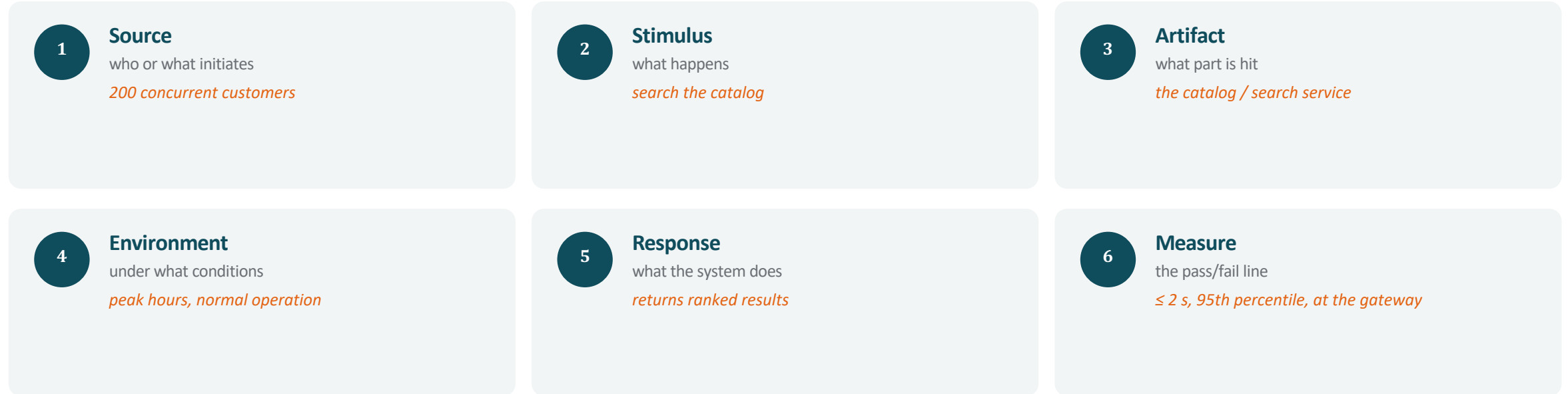
v3 — a scenario

“During peak hours, with 200 customers browsing concurrently, a catalog search returns ranked results in ≤ 2 s at the 95th percentile, measured at the API gateway.”

Testable, bounded, honest about load and measurement point.

THE RULE A quality requirement is done when a tester could fail it — same Standard of Quality you used on stories, applied to qualities.

Six parts make a quality claim testable



Read as one sentence — a performance scenario:

“During peak hours, 200 concurrent customers searching the catalog get ranked results in ≤ 2 s (95th percentile, measured at the gateway).”

THE RULE Six boxes, one sentence, one pass/fail line — that is a quality attribute scenario.

Quality attribute scenario — the blank form

1 • Source	<i>who or what initiates</i>	
2 • Stimulus	<i>what happens</i>	
3 • Artifact	<i>which part is hit</i>	
4 • Environment	<i>under what conditions — normal? degraded? peak?</i>	
5 • Response	<i>what the system does</i>	
6 • Measure	<i>number + unit + percentile + where measured</i>	

Read it back as ONE sentence: “During _____, _____ causes _____ to _____, measured as _____.” Then score it: importance H / M / L · difficulty H / M / L.

THE RULE Three scenarios is the working minimum: one performance, one availability, one security or cost.

Availability, worked — the third-party feed dies

WEAK — HIDES THE DECISION

“The system shall be highly available (99.9%) even if external providers fail.”

- Which functions? All of them — including trading on old prices?
- 99.9% of what — the year? peak hours matter more.
- No stated behavior DURING the failure — the actual design question.

STRONG — FORCES THE DECISION

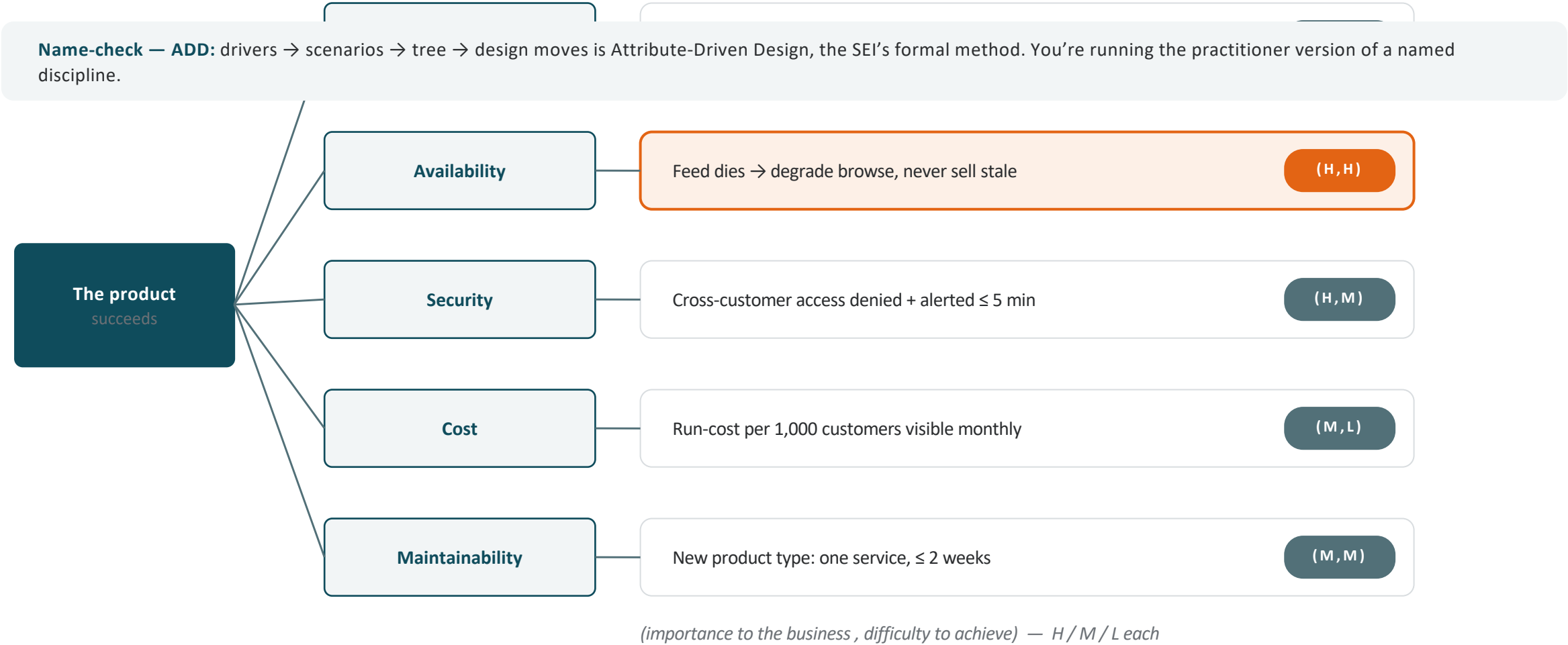
Source: third-party price/stock feed · Stimulus: feed unreachable or stale > 60 s ·
Artifact: whole platform · Environment: peak hours

Response: browse & search continue on last-good data, labeled “delayed”; ordering suspends with a clear message; ops alerted.

Measure: browse ≥ 99.9% of peak hours · zero orders on prices older than 60 s · alert within 2 min.

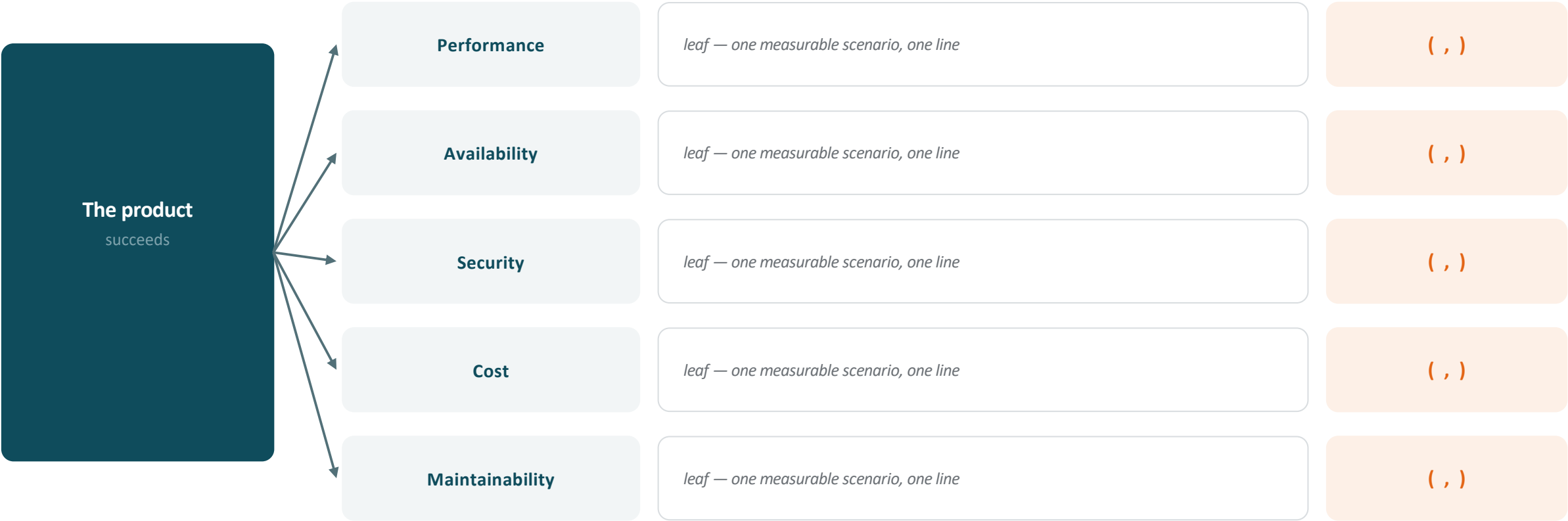
THE RULE A good availability scenario states behavior DURING failure — degraded browse, suspended ordering, labeled staleness.

The utility tree — drivers down to testable scenarios



THE RULE The (H,H) leaves are your architecture’s to-do list — and Chapter 6’s review agenda.

Utility tree — the blank frame



(importance , difficulty) — H / M / L each

THE RULE Circle every (H,H) leaf — those are the promises your design gets judged against.

Every quality claim is measurable — now pick a shape that can deliver

✓ Compare the five quality attributes and their tensions

✓ Write six-part scenarios a tester could fail

✓ Build and score a utility tree from business drivers

✓ Know where review effort goes: the (H,H) leaves

NEXT · CHAPTER 4

Monolith? Services? Event-driven? Serverless?

The styles tour, the cloud building blocks, DDD — and the microservices deep dive your scenarios will judge.

04

Styles, DDD & Microservices — and the Cloud

Choosing a shape is four questions, asked in order: what are the options, where are the seams, how do the pieces talk and fail, and how separately should they run — on what. Cloud vocabulary first, then each question in turn, microservices in depth — and the lab assembles your four answers into a candidate.

THE COURSE

- 1 Mining the Backlog — Forces & ASRs
- 2 Use Case Modeling — Behavior & a First Candidate
- 3 Quality Attributes, Scenarios & the Utility Tree
- 4 Styles, DDD & Microservices — and the Cloud**
- 5 Documenting the Solution
- 6 Analyzing & Reviewing — the ATAM Discipline
- 7 Hardening, Hand-offs, Cost & Effort

Every architecture is a bet

A style is a bet that a structure will deliver your scenarios at a price you can pay.

Your stake: your top scenarios — quick search at peak, survive the feed dying — the (H,H) leaves from your tree.

Your judges: the compliance committee (records, controls) and the technology board (“why isn’t this standard?”).

Your temptation: the style everyone names before hearing the problem.

THIS CHAPTER

- The cloud’s five building blocks
- Five most-used styles, fast
- Microservices, properly: boundaries, data, comms, failure
- When NOT to — said out loud
- What each piece runs on

THE RULE Styles are answers. Your utility tree is the question — keep it on the table all chapter.

Five building blocks — all any design needs to name



Compute

Something that runs your code: a whole machine, a container, or just a function.

e.g. runs your search & checkout logic



Data

Managed databases, object storage, warehouses — kept alive, backed up, patched for you.

e.g. products, orders, audit records



Messaging

Queues and topics — letters between parts, not phone calls. The sender doesn't wait.

e.g. "OrderPlaced" → records, receipts



Edge & network

The front door: load balancer, API gateway, CDN, DNS. One entrance, guarded.

e.g. phone + desktop hit one API



Identity & ops

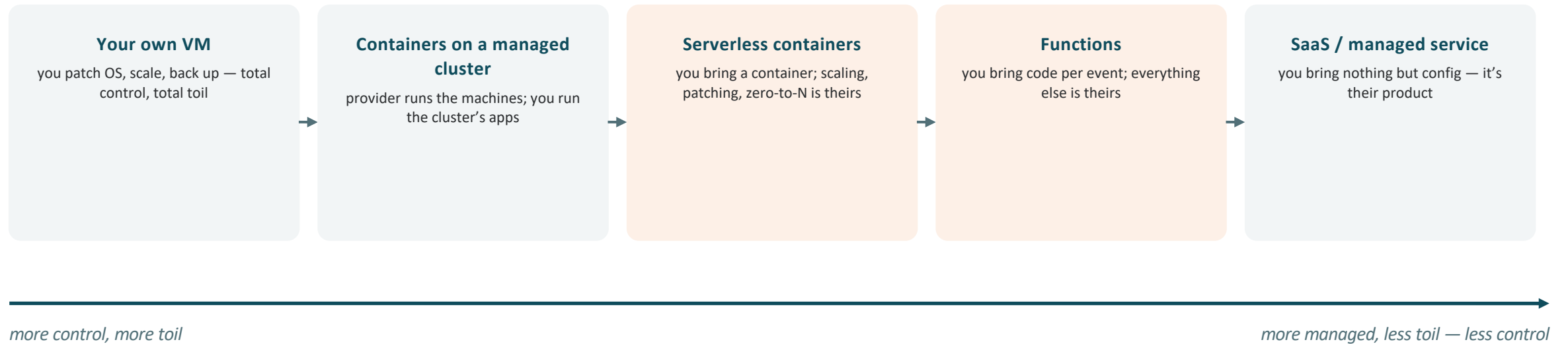
Who may do what (IAM) — plus the meters: logs, metrics, alerts.

e.g. auditors will ask for both

That's the whole alphabet. Every diagram you draw today is these five, arranged. If a box on your design isn't one of them (or a person, or someone else's system), ask what it really is.

THE RULE Name parts by building block, not by brand — products change, the five blocks don't.

Managed vs self-run — control for toil



Default for a small product team: as managed as your constraints allow.

Buy back control only when a scenario demands it — a latency floor, special workloads, or a portability mandate from your technology board. Every step left costs engineer-hours forever.

THE RULE The managed↔self-run dial is set per piece, per scenario — never once for the whole system.

Layered monolith

SWEET SPOT

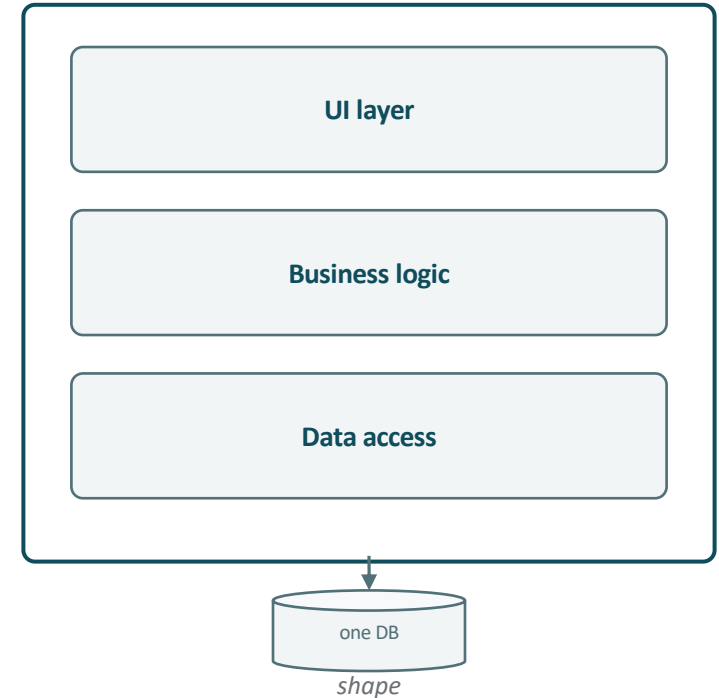
Small team, simple domain, fastest possible start. One build, one deploy, one thing to debug.

FAILURE MODE

Growth: layers don't stop features tangling. Scaling means cloning the whole thing; one bad deploy takes all of it down.

TYPICAL FIT

A legitimate way to START most products — but audit records, third-party adapters and the money path all scale and fail differently. Watch that.



Modular monolith

SWEET SPOT

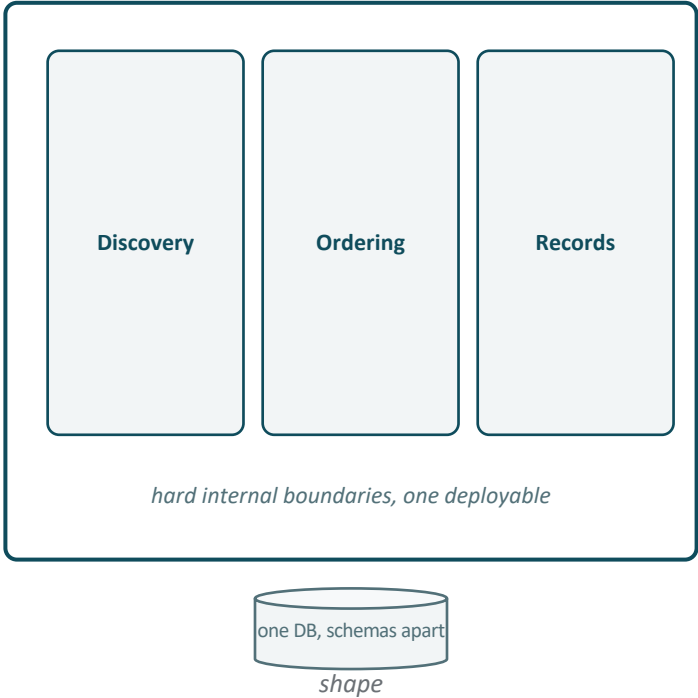
Monolith simplicity, service-style boundaries. The strongest default for one team — and the best launchpad if you split later.

FAILURE MODE

Boundaries live on discipline alone: one “quick” cross-module import at a deadline, and they’re gone. Nothing physical stops it.

TYPICAL FIT

The serious rival to microservices for any one-team release. Hold this thought — it returns on the “when NOT to” slide.



Event-driven

SWEET SPOT

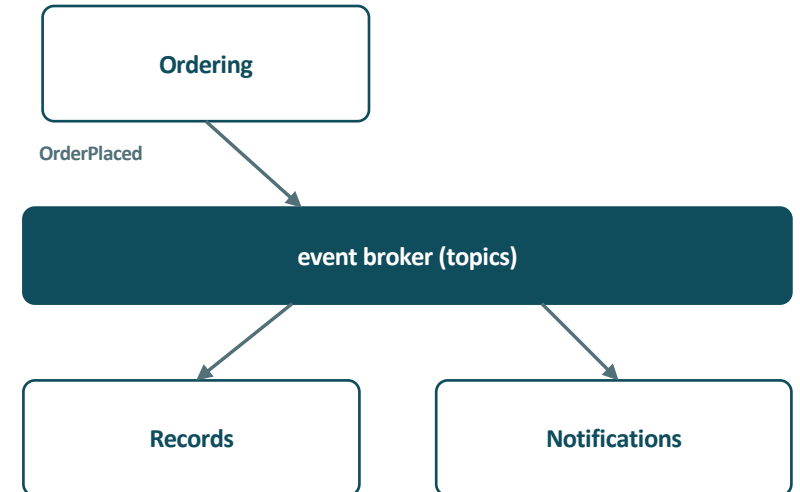
Parts react to facts (“OrderPlaced”) instead of calling each other. Spikes absorbed by queues; the event log is a natural audit trail.

FAILURE MODE

Nobody can say “what happens next” by reading one file. Debugging is archaeology; eventual consistency surprises the unprepared.

TYPICAL FIT

Confirmations, receipts and audit records LOVE this. The transactional step itself wants stricter ordering — mix, don’t worship.



shape

Service-oriented (SOA)

SWEET SPOT

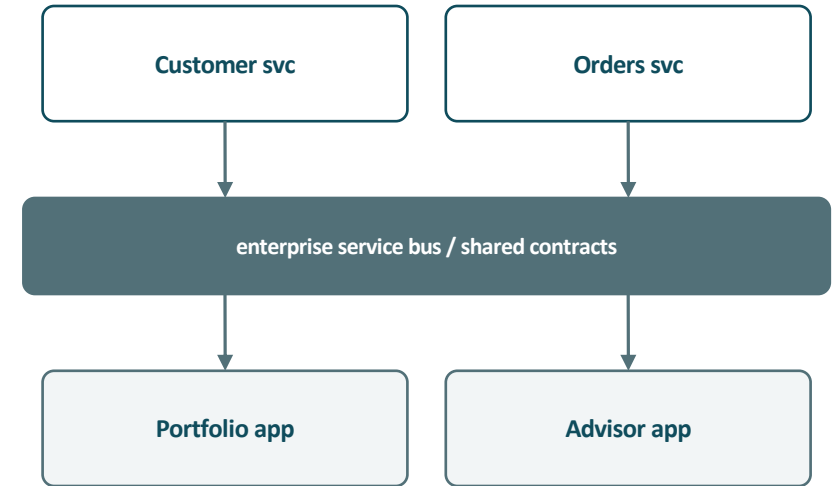
Coarse business services shared across MANY applications — enterprise integration, reuse, central governance. How large enterprises already think.

FAILURE MODE

The shared bus and canonical data model become the choke point: every change negotiated, every outage shared.

TYPICAL FIT

Context, not blueprint: in a large firm, the board may ask your product to consume existing enterprise services. Plan for that seam.



shape

Serverless

SWEET SPOT

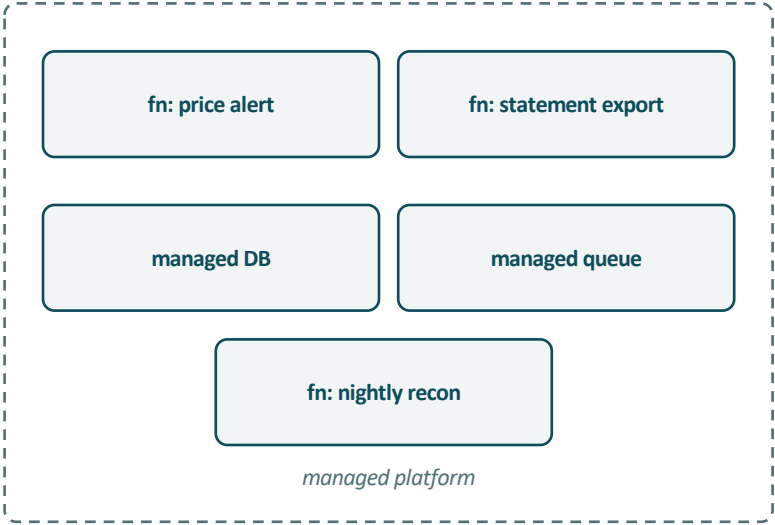
Code runs only when called; idle costs ~nothing; provider does the ops. Perfect for spiky, event-shaped, glue-and-edges work.

FAILURE MODE

Hundreds of tiny pieces with no map; cold starts on latency-critical paths; per-request pricing stings at steady high volume.

TYPICAL FIT

Ideal for the edges — alerts, exports, nightly jobs. The core transaction path can run this way too; reason about latency and cost first.



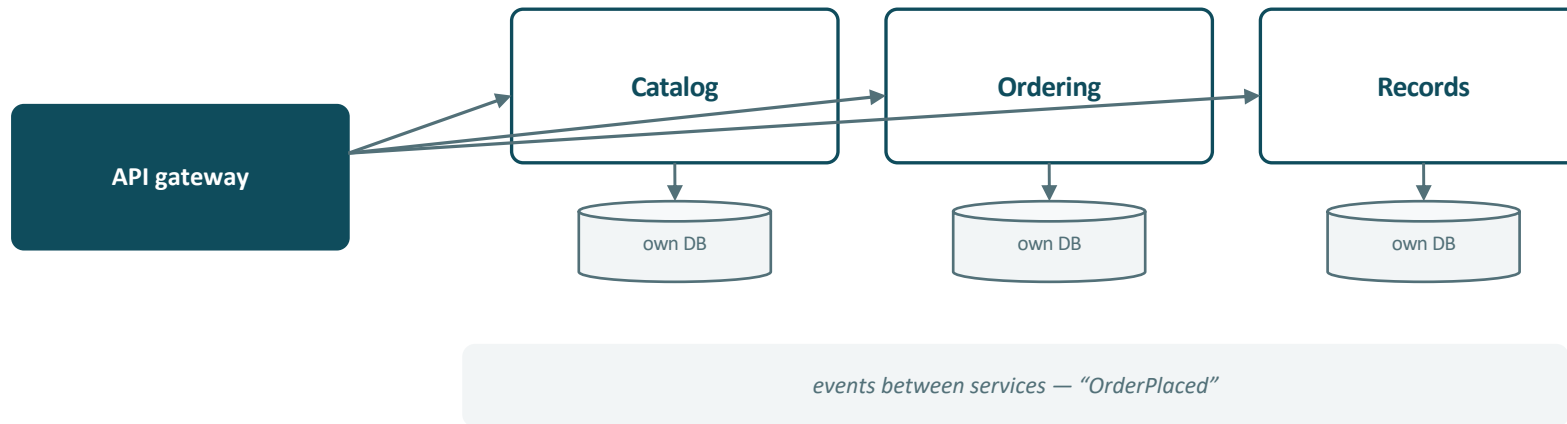
shape

When to reach for which

Style	Reach for it when...	It bites when...
Layered monolith	one team, simple domain, speed to first release	the tangle: layers don't stop coupling
Modular monolith	one team, real domain, want boundaries + simplicity	discipline decay — boundaries are social, not physical
Event-driven	audit trails, spikes, many downstream reactions	hard to trace; eventual consistency surprises
Service-oriented	many apps sharing enterprise capabilities	the shared bus/model becomes the bottleneck
Serverless	spiky or event-shaped load, tiny ops team	sprawl, cold starts, per-request cost at volume
Microservices	several teams, parts that scale/change/fail differently	operational tax on every service, forever

THE RULE Styles mix. Real systems are hybrids — pick per PART, against the tree, not one badge for the whole system.

Small services, real independence — that's the whole point



Three defining properties

- Split by business capability — Catalog, Ordering, Records... never “UI service, DB service”.
- Each owns its data. Others ask via API or events — never reach into its store.
- Independently deployable — ship one without rehearsing the others.

It's an organizational pattern as much as a technical one: independence pays when TEAMS need it.

THE RULE If two “services” must always deploy together, you have one service and a network bill.

DDD-lite: bounded contexts draw the lines

Ubiquitous language

Inside a boundary, one word means one thing. “Order” means one thing to Sales and another to Fulfilment — one word, two contexts, and your domain sketch already knew.

The rule of thumb

One bounded context → one service candidate. Candidate — the tree and the operational bill still get a vote before it becomes a service.

Bounded context

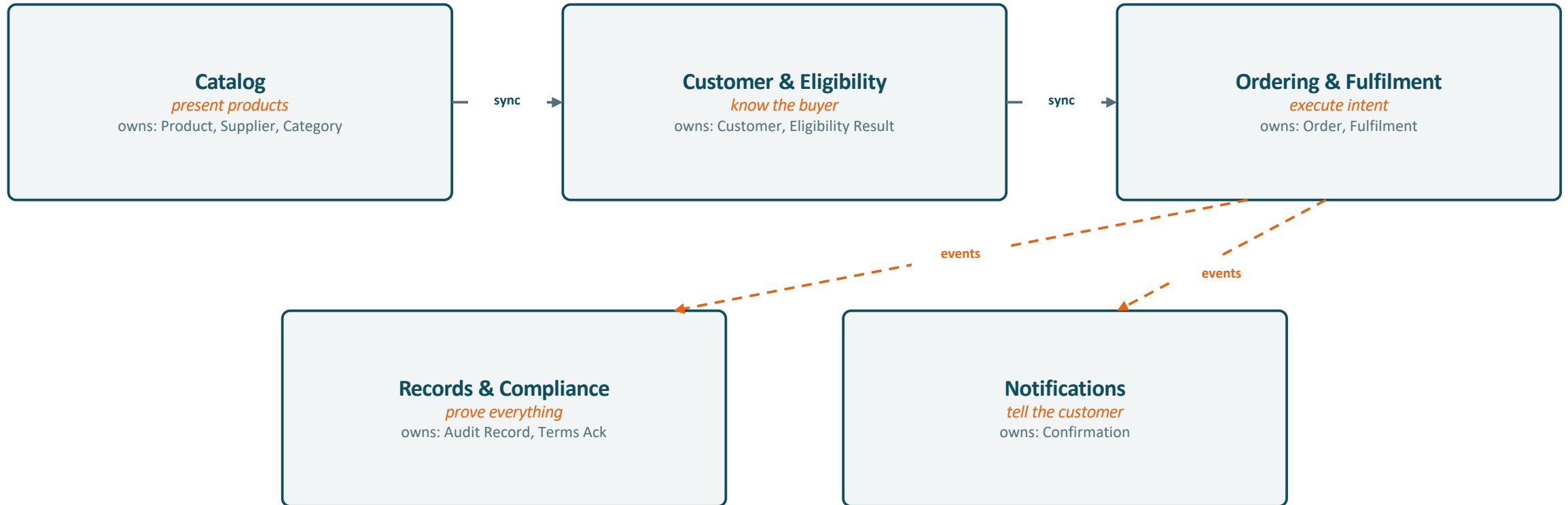
The line where language changes. To the catalog, a product is a THING TO PRESENT; to ordering, a THING TO SELL; to finance, a THING TO PROVE. Three meanings → three contexts.

The wrong starting points

Database tables (structure ≠ meaning) · one-service-per-noun (“Customer service, Product service...” — a distributed monolith with extra steps) · org chart alone.

THE RULE Where the language changes, a boundary wants to exist — that’s the whole trick.

A context map, worked — five candidates

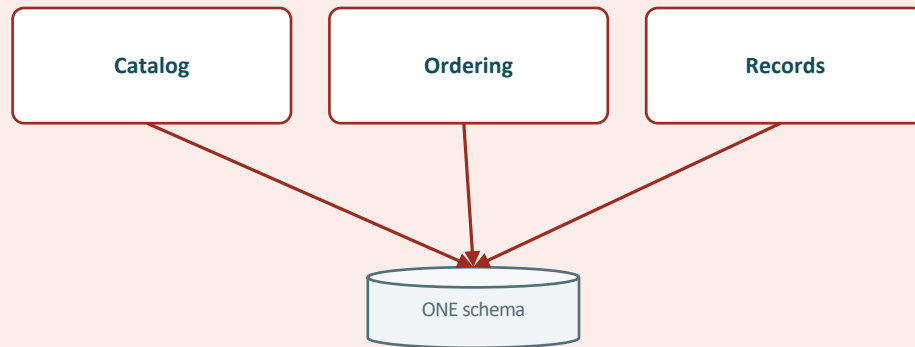


Honest footnote: five is the cleanest cut, not the only one. A one-team Release 1 may merge to three — or run all five as modules in a modular monolith. Boundaries first; separate deployment is a second, later decision.

THE RULE Sync where you must wait (eligibility gates checkout) — events where you mustn't (records, receipts).

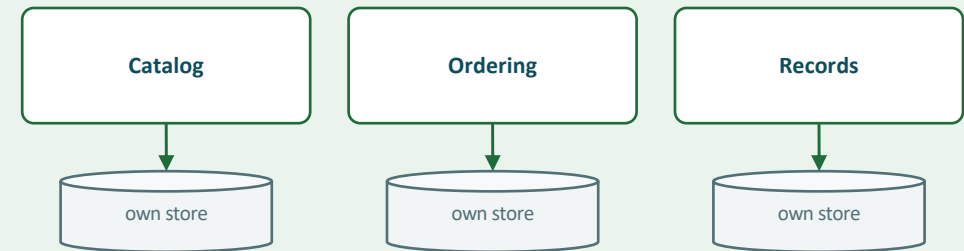
One owner per fact — database-per-service

THE SHARED-DATABASE TRAP



Every schema change is a three-team negotiation; every slow query is everyone's outage. Independence: gone.

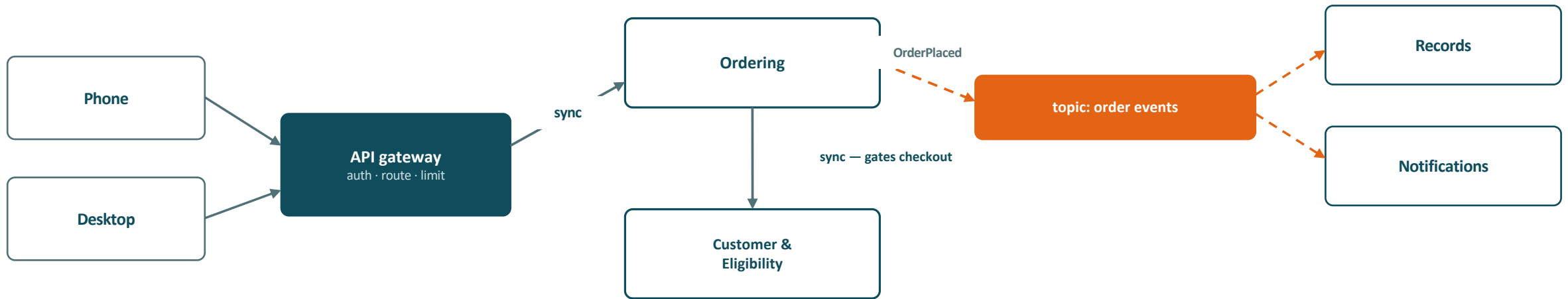
ONE OWNER PER FACT



- Need someone else's fact? Ask their API, or carry it on the event.
- Only Records writes audit records — Ordering emits the fact, Records makes it provable for the auditors.

THE RULE “Who owns this fact?” must have exactly one answer — your CH1 domain nouns are the ownership ledger.

Sync where you wait, async where you don't — one gate at the front



The test, per edge — “Can this step finish later?” No → sync call. Yes → publish an event and move on. The customer waits for the eligibility check; the audit record can lag by milliseconds.

API-first (name-check) — Write the contract (OpenAPI) before the code. Teams build against the contract in parallel — the contract, not the meeting, is the agreement.

Why one gateway — Two clients, one door: authenticate once, rate-limit once, version once — and measure your performance scenario’s “p95 at the gateway” in exactly one place.

THE RULE Every arrow you draw is a promise about waiting. Label each one sync or async — no unlabeled arrows after today.

Timeouts, retries, circuit breakers — the seatbelts



Timeout

Never wait forever. Every sync call gets a deadline; a hung feed must not hang the buy page.



Retry + backoff

Try again briefly, with jitter — but only idempotent operations. Retrying a maybe-executed purchase double-buys; an idempotency key makes it safe.



Circuit breaker

Failing repeatedly? Stop calling. Fail fast, serve the fallback, let the feed breathe, probe gently.

WORKED EXAMPLE · THIRD-PARTY FEED CALL POLICY (ILLUSTRATIVE NUMBERS)

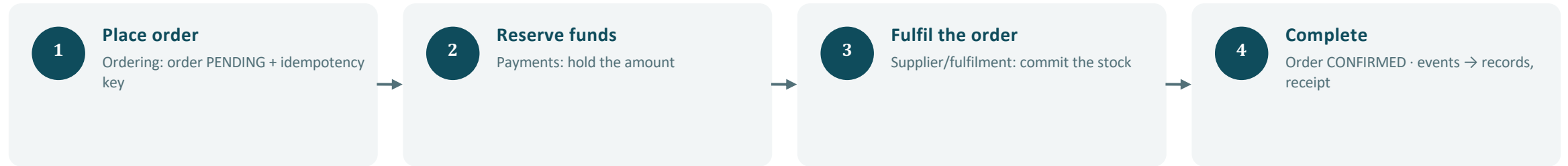
`timeout 800 ms · 2 retries, jittered backoff · breaker: opens after 5 failures in 30 s`

`fallback: last-good price if  $\leq 60$  s old, labeled "delayed" — otherwise SUSPEND ordering, keep browse alive`

...which is exactly your availability scenario, translated into settings. Scenarios become configuration.

THE RULE Every sync arrow gets all three seatbelts — the design question is only what the fallback IS.

The purchase saga — no shared transaction, so compensate



STEP 3 FAILS → COMPENSATE BACKWARDS

release the fund hold → mark order FAILED (kept, not deleted — auditors read failures too) → tell the customer honestly

Nothing is half-done; the trail is complete. That is UC-3's extension 7a, answered.

Eventual consistency, honestly: for a few hundred milliseconds the order is CONFIRMED and the receipt isn't sent. Design what the customer sees at each stage — "processing" is a legitimate state, silence is not.

THE RULE Across services there is no shared COMMIT — every multi-service write needs its undo story written first.

Observability — or “where did order 4711 go?”



Logs

What happened, in words. Structured, with the order ID on every line.



Metrics

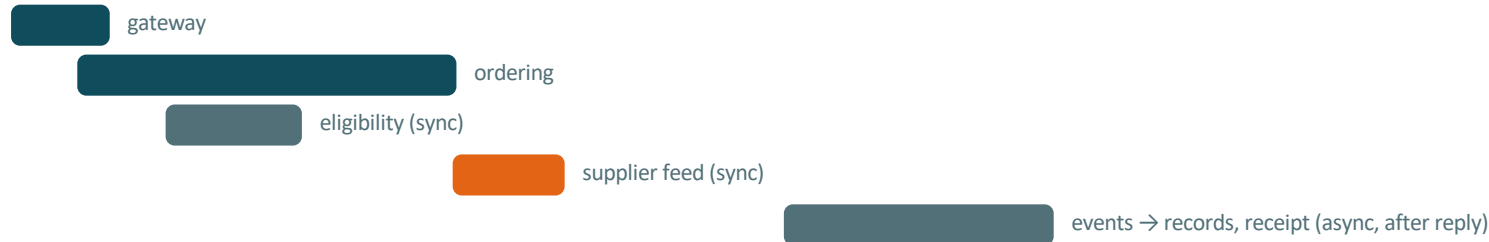
How much, how fast, how often. Your scenario’s “p95 ≤ 2 s” only exists if the gateway emits it.



Traces

One request’s journey stitched across services by a correlation ID born at the gateway.

trace: POST /orders · order 4711 · total 640 ms



THE RULE Every measure in your QAS list must be **EMITTED** by some component — decide which, at design time.

The bill — and when a modular monolith wins

THE TAX, PER SERVICE, FOREVER

- A pipeline, a deploy, monitoring, an on-call answer
- A network hop on every call that used to be a function call
- Sagas + idempotency where a transaction used to do
- Debugging = distributed archaeology (traces mandatory)
- And the cloud bill line per service (QAS-C1 watches)

PAY IT ONLY FOR REAL DRIVERS

- Several TEAMS need to ship without meetings
- Parts truly scale differently (search vs nightly recon)
- Parts fail differently (feed adapter must die alone)
- Parts change at different speeds (catalog weekly, records rarely)

One team on Release 1? A modular monolith with clean seams — or three services — is a STRONG answer. Five is a bet on future teams; if you make it, say so in the review.

THE RULE Microservices buy independence and charge operations. If nobody needs the independence, don't pay the tax — boundaries are free, deployment isn't.

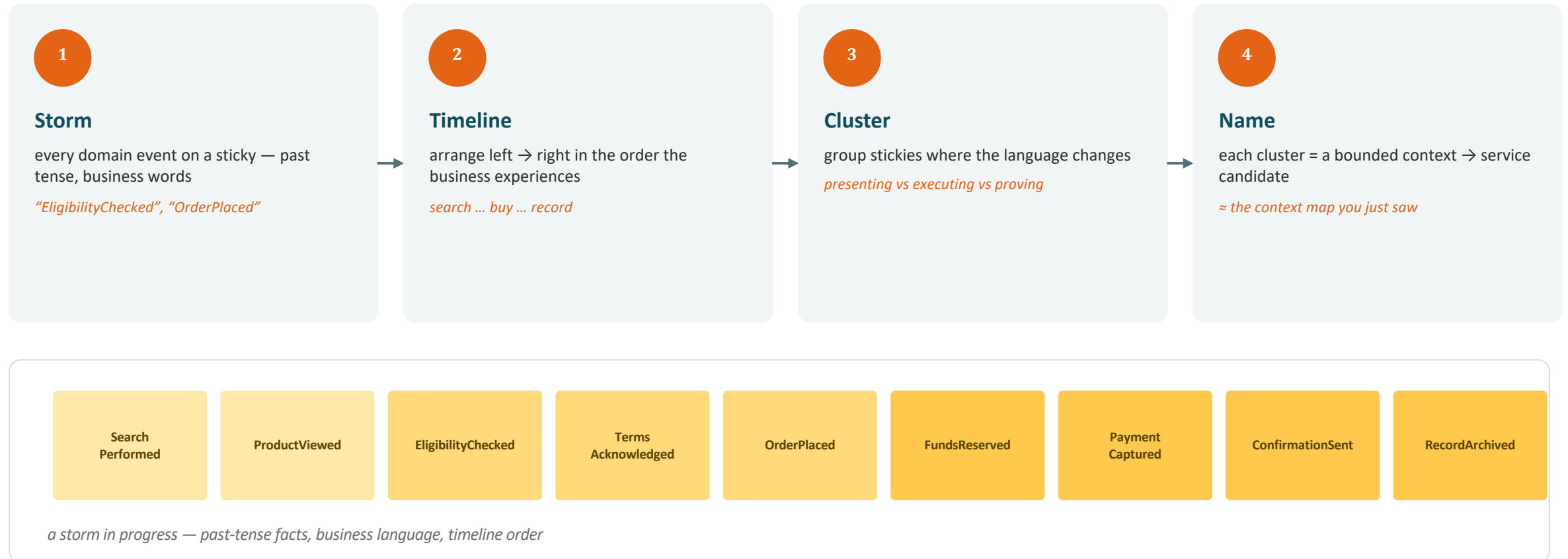
Decide per service: what does each piece run on?

Piece	Its nature	Runs on (vendor-neutral)
Catalog / search	stateless request service, spiky at peak	serverless container, scale-to-N
Ordering & payments	stateless requests, latency floor on checkout	serverless container + warm minimum instances
Third-party feed adapter	always listening to a stream	small always-on worker service
Records & compliance	append-heavy writer, must never lose	serverless container + managed DB + object archive
Notifications	event-shaped bursts, latency-tolerant	functions on events
All databases & the event bus	state — the part you least want to babysit	managed services, full stop

Each row is an ADR waiting to be written. The dial is set per piece, justified against the tree (latency floor → warm instances; cost QAS → scale-to-zero on quiet pieces). Chapter 5 writes one down.

THE RULE “What does it run on?” is answered per service, against scenarios — never once, in the first meeting, for everything.

Event storming (lite) — the wall finds the boundaries



THE RULE Past tense is the trick: "OrderPlaced" is a fact the business recognizes — facts cluster into contexts, features don't.

You chose a shape. Now write it down before lunch does

- ✓ Q1 · What are the options — a style-family shortlist, judged by the tree
- ✓ Q2 · Where are the seams — a context map, one owner per fact
- ✓ Q3 · How pieces talk & fail — labeled edges, seatbelts, sagas, observability
- ✓ Q4 · How separate, on what — the bill argued, runs-on set per piece
- ✓ Your four answers, stacked = the candidate architecture on your wall

NEXT · CHAPTER 5

A wall of stickies convinces nobody.

Views for every audience, one sequence that exposes the waits, a deployment map with real product names — and your first ADR.

05

Documenting the Solution

Your design so far lives on a wall and in your heads. This chapter turns it into the four artifacts reviewers actually read: views at three zoom levels, a sequence diagram that exposes every wait, a deployment map with real product names — and your first architecture decision record.

THE COURSE

- 1 Mining the Backlog — Forces & ASRs
- 2 Use Case Modeling — Behavior & a First Candidate
- 3 Quality Attributes, Scenarios & the Utility Tree
- 4 Styles, DDD & Microservices — and the Cloud
- 5 Documenting the Solution**
- 6 Analyzing & Reviewing — the ATAM Discipline
- 7 Hardening, Hand-offs, Cost & Effort

The review board reads diagrams, not your head



Different readers, different zooms

The compliance committee wants boundaries and record flows. Engineers want containers and contracts. Nobody wants everything at once.



Undocumented = re-litigated

The choice you can't show gets re-argued in every meeting until someone writes it down. Documentation is how decisions stay decided.



The everything-diagram lies

40 boxes and every arrow = nothing communicated. What you leave OUT of each view is the actual skill.

THE RULE Documentation is not homework after design — choosing what each audience sees IS design.

Context → containers → components (C4-style)

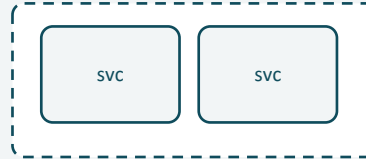


1 · Context

the system as ONE box among people & other systems

read by: committees, sponsors, everyone

answers: what is this and who touches it?

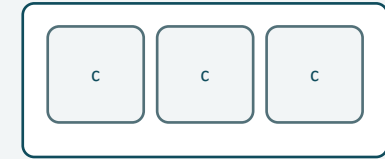


2 · Container

the separately RUNNING pieces (apps, services, DBs, bus)

read by: engineers, ops, reviewers

answers: what do we build, deploy, scale?



3 · Component

inside ONE container: its major parts

read by: the team that owns it

answers: how is this one built? (draw only where it earns it)

Two traps, named — and where the old diagram names went:

“Container” = a thing that RUNS, not Docker. The 4+1 view model is this idea’s classic ancestor — same instinct, more ceremony; C4 is today’s default.

A “system diagram” = the context view · an “architecture diagram” = the container view · the NETWORK diagram (VPCs, subnets, firewalls) lives one level BELOW the deployment view — platform engineering’s layer, routed as a Chapter 7 hand-off.

THE RULE One zoom per audience, one question per view — mixing zoom levels is how diagrams start lying.

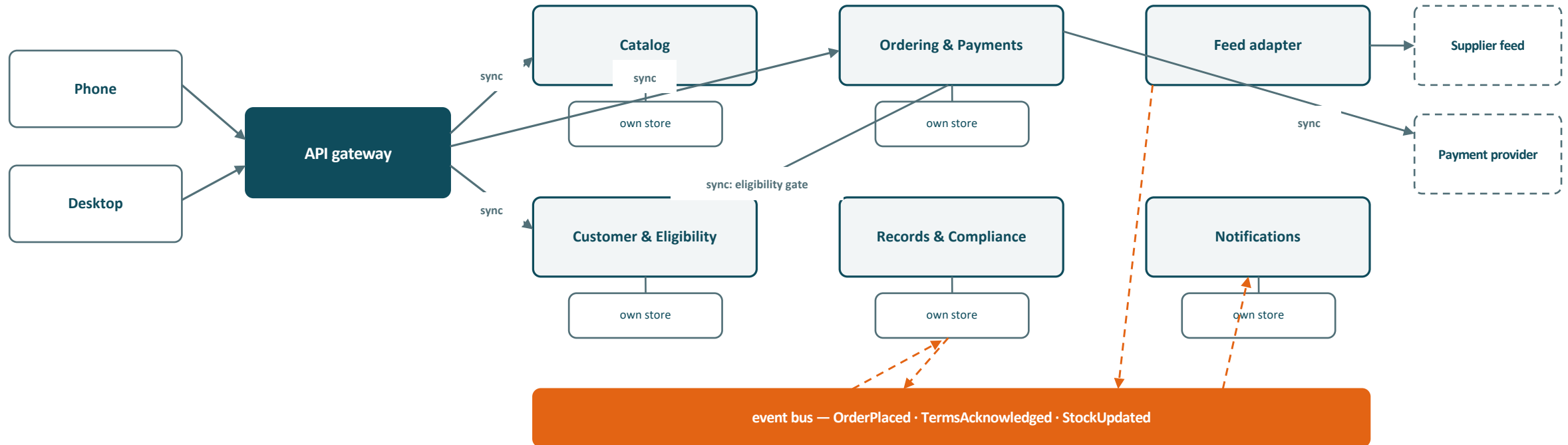
Context view, worked — one box, five neighbors

Audience check: the compliance committee reads THIS before the charter. No services, no databases, no product names — on purpose. Every arrow is a business sentence.



THE RULE If a committee member asks “what’s a container?”, you brought the wrong zoom.

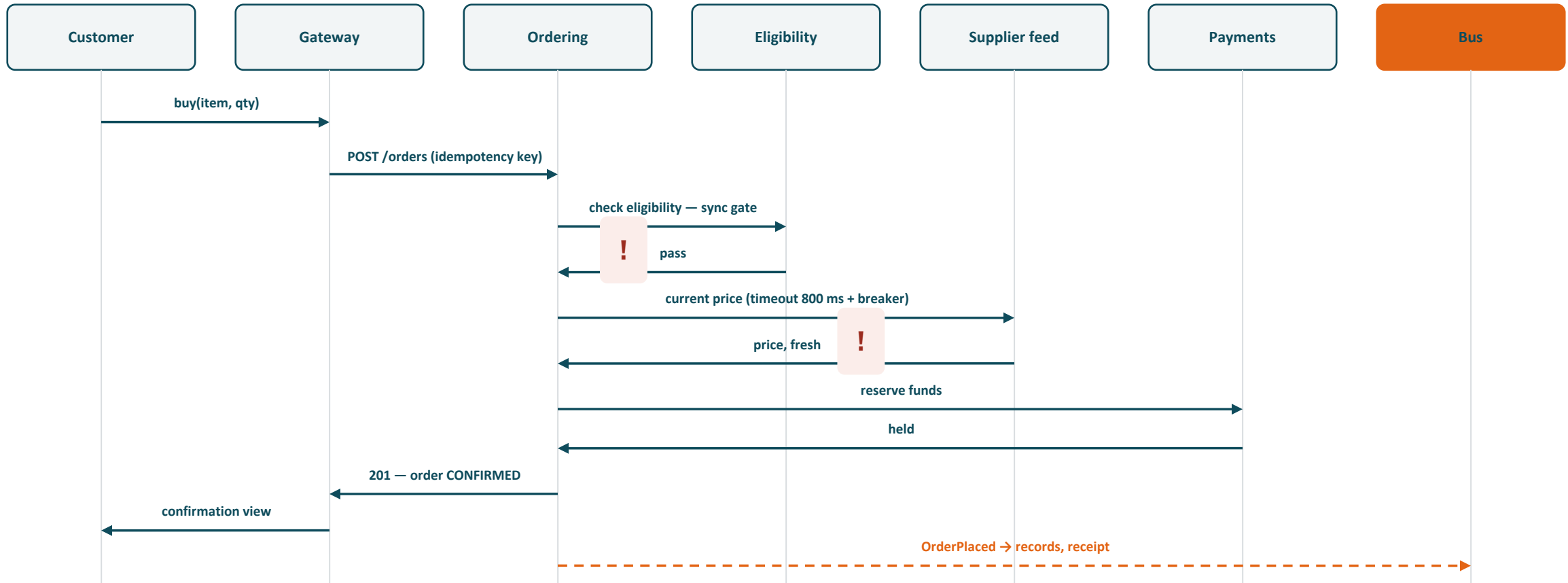
Container view, worked — the pieces that run



Reading order for a reviewer: front door → sync spine (gateway, services, gates) → async nervous system (the bus) → externals at the edge, dashed. Own store under every service — the ownership rule, visible.

THE RULE The container view is your context map made deployable — if an arrow has no sync/async label, it isn't finished.

Sequence, worked: “customer places an order” — every wait, visible



Solid = the customer is waiting. Dashed = nobody waits. The two ! marks — the external price call and the in-path compliance gate — are tomorrow’s incidents and Chapter 6’s trade-off points.

Deployment view — name what each container runs on

Container	Runs on (neutral)	On GCP	AWS · Azure
Web & phone clients	static app + responsive web	Firebase Hosting / Cloud CDN	Amplify · Static Web Apps
API gateway	managed API front door	API Gateway (or Apigee)	API Gateway · API Management
Services (Catalog, Ordering, Records...)	serverless containers	Cloud Run	Fargate · Container Apps
Feed adapter	small always-on worker	Cloud Run (min instances) / GKE	ECS service · AKS
Event bus	managed pub/sub topics	Pub/Sub	SNS+SQS · Service Bus
Service stores	managed relational / document DB	Cloud SQL · Firestore	RDS/DynamoDB · Azure SQL/Cosmos
Audit archive	immutable object storage + lock	Cloud Storage (retention lock)	S3 Object Lock · Immutable Blob
Identity & monitoring	IAM, customer auth, metrics/logs/traces	Cloud IAM · Identity Platform · Cloud Monitoring	IAM/Cognito · Entra ID/Monitor

THE RULE Neutral names in the design, product names in the deployment view — the design survives the vendor, the map makes it buildable.

An ADR: one page, four parts, dated

1

Context

The forces at that moment: scenarios, constraints, politics, deadlines. Written so a stranger in six months gets it.

2

Options considered

The 2–3 real alternatives, each with its honest upside. If there was no alternative, it wasn't a decision.

3

Decision

What you chose — one sentence, active voice: “We will...”

4

Consequences

What gets better AND worse. The costs you accepted knowingly. This is the half reviewers read first.

WHEN A DECISION EARNS AN ADR

- A newcomer would ask “why?” in six months
- It closes a door that’s expensive to reopen
- A reviewer challenged it — write the answer down once

Numbered, dated, never deleted — superseded ADRs point at their successor.

THE RULE An ADR is the answer you never have to give twice.

ADR-007: caching third-party feed prices

POOR — A VERDICT, NOT A RECORD

“Decision: We will cache prices in Redis for performance.”

- Why? What forces? (no context)
- Compared to what? (no options)
- Stale prices on ORDERS?! (no consequences — the one question that matters is unanswered)
- Product name in the headline; decision welded to a tool.

GOOD — ATTACKABLE, THEREFORE USEFUL

Context: the performance scenario wants search ≤ 2 s; the availability scenario forbids selling on stale prices; provider SLA unconfirmed.

Options: (a) call feed on every view — freshest, slowest, fragile · (b) cache with TTL — fast, staleness risk · (c) local price replica — fast, heavy to sync.

Decision: cache prices for BROWSE with a 60 s TTL, labeled “delayed”. The ORDER path always fetches live — no exceptions.

Consequences: browse survives feed wobble (+) · customers may see 60 s-old prices, labeled (accepted) · order path keeps a hard dependency on the feed — mitigated by breaker + suspend (known, monitored).

THE RULE A good ADR is one a reviewer can attack — vague records are invulnerable and worthless.

Architecture decision record — the blank page

ADR-__ · title · date · authors · status: proposed / accepted / superseded by ADR-__

1 · Context — the forces

Which scenarios, constraints and dependencies are pressing on this decision right now?

2 · Options considered

Each with a GENUINE upside and a genuine downside. Three rows. No strawmen.

3 · Decision

“We will ...” — one sentence, active voice, including the hard line it draws.

4 · Consequences

What gets better (+) | what gets worse, the cost you accepted knowingly (–)

THE RULE A record with only upsides is advertising. Name the cost you accepted — reviewers read that part first.

Documented — which means attackable. Good.

✓ Draw context, container and component views for the right audiences

✓ Sequence a critical flow and mark where it hurts

✓ Ground every container in a runs-on choice with real names

✓ Write an ADR a reviewer can attack

NEXT · CHAPTER 6

Your design is now evidence.

Sensitivity points, trade-offs, risks — and the ATAM review that hunts them. Then: defending under challenge, without flinching.

06

Analyzing & Reviewing — the ATAM Discipline

Your design is documented — now it goes on trial. The vocabulary of analysis (sensitivity points, trade-off points, risks, non-risks), the ATAM review that hunts them across platform, data, AI, security and operations at once — and the craft of defending a design without defending your ego.

THE COURSE

- 1 Mining the Backlog — Forces & ASRs
- 2 Use Case Modeling — Behavior & a First Candidate
- 3 Quality Attributes, Scenarios & the Utility Tree
- 4 Styles, DDD & Microservices — and the Cloud
- 5 Documenting the Solution
- 6 Analyzing & Reviewing — the ATAM Discipline**
- 7 Hardening, Hand-offs, Cost & Effort

Wednesday, 14:00 — the pre-charter review

Across the table:

the compliance chair (records, controls, no surprises) · two review-board CTOs (“why isn’t this our standard stack?”) · a business head who thinks the whole idea is a mistake

First question:

“Walk me through what the customer sees when your third-party feed dies at 2 p.m. on your busiest day.”

You have the sequence diagram. You have ADR-007. This chapter is how you use them.

WHY REVIEWS PAY

A flaw found today costs a diagram.

The same flaw in production costs your busiest day — and the committee’s trust.

A review is the cheapest failure you will ever be offered. Take it.

THE RULE The review is not the obstacle to your design shipping — it is the reason it deserves to.

Sensitivity points and trade-off points

SENSITIVITY POINT

A place where ONE quality hangs on ONE decision.

Turn the dial, the quality moves. The cache TTL — at 10 s, browse dies with the feed; at 10 min, customers see ancient prices.

TRADE-OFF POINT

One decision, SEVERAL qualities — pulling opposite ways.

The same TTL: availability up, freshness down. The eligibility gate: compliance up, checkout-latency down. Someone must choose — visibly.

More examples — say them in review language:

Single region vs two — trade-off: availability & disaster posture ↔ cost & complexity (the cost scenario watches)

Managed event bus — trade-off: ops toil ↔ portability — the lock-in question a review board WILL ask

Warm instances on Ordering — sensitivity: checkout-path p95 hangs on the minimum-instance setting

THE RULE Sensitivity: one dial, one quality. Trade-off: one dial, a tug-of-war. Reviews live at the trade-offs.

Risks and non-risks — both get written down

RISK — CONDITION → CONSEQUENCE

R-1

BECAUSE the supplier-feed SLA is unconfirmed AND the order path calls it live, a feed outage during peak hours suspends ALL ordering — with no committed time-to-restore from the vendor.

Owner: vendor management · Evidence: ADR-007, sequence ! mark · Status: OPEN, blocking charter sign-off

Not “the feed worries us” — a testable claim with a consequence and an owner.

NON-RISK — EXAMINED AND PASSED

NR-1

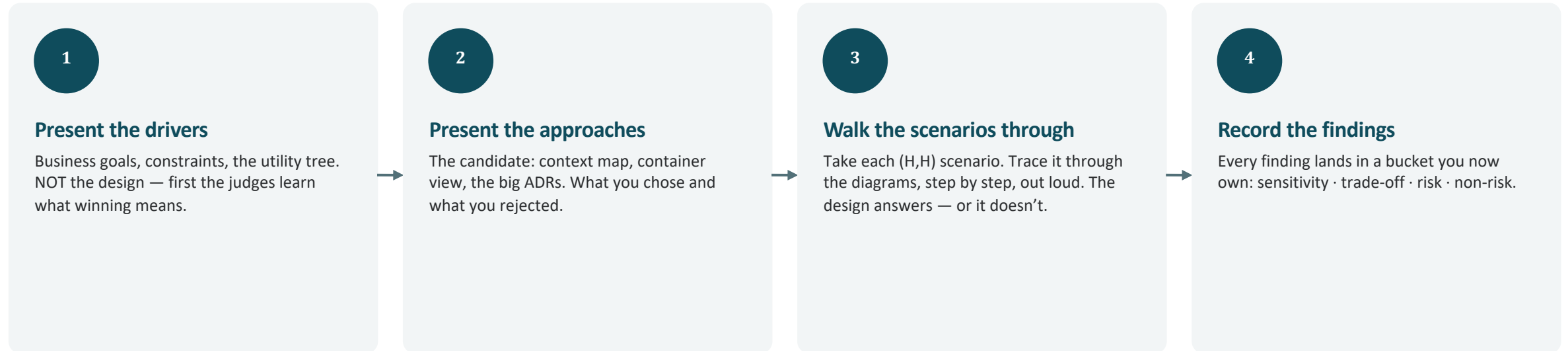
Serving phone and desktop from one responsive web app + one API is NOT a risk for Release 1: existing customers, standard patterns, no native-app features required.

Valid WHILE: Release 1 targets existing customers only. If a native app or new-customer onboarding enters scope, this reopens.

Written down so the same worry isn’t re-raised in every meeting — that’s the point.

THE RULE A risk names condition, consequence and owner. A non-risk names its expiry condition. Both are review outputs.

The review that hunts them: ATAM in four moves



Name-check, with respect: ATAM — the Architecture Tradeoff Analysis Method (SEI).

The full method is a facilitated multi-day engagement. What you're running today is its engine, sized for a working review — the same engine your QAS and utility tree were built for. It all connects here.

THE RULE Drivers before design, scenarios before opinions, findings in four named buckets — that's a review, not a meeting.

Platform, data, AI, security, operations — together



Platform

Regions & zones: what fails together?
Quotas? The lock-in trade-off —
answered, not dodged?



Data

One owner per fact? Retention &
deletion rules? Audit archive
immutable — provably?



AI

Nothing in R1? Say so, record the non-
risk. But: does today's data retention
keep tomorrow's "recommended for
you" possible?



Security

AuthN/Z at the gateway AND
between services? PII where? (STRIDE
is the specialist's deep version —
hand off.)



Operations

Who wakes up? Can you deploy
Tuesday 14:00? Does every QAS
measure get emitted?

The anti-pattern: five separate review meetings, five contradicting verdicts — and the cross-lens findings (cache TTL is platform AND data AND security) fall in the gaps between them. One table, five question sets, one findings list.

THE RULE Run five lenses over one table — the interesting findings live where the lenses overlap.

The availability scenario walks through the design

Feed goes silent 60 s	breaker opens; browse serves last-good, labeled “delayed”	✓ answered — ADR-007 + breaker policy
Customer mid-checkout	order path refuses stale price; ordering suspends with message	✓ answered — hard line in ADR-007
How stale is “last-good”?	TTL 60 s — turn it up, availability rises, truth decays	SENSITIVITY + TRADE-OFF — recorded
How long can this last?	...no committed vendor restore time exists	RISK R-1 — owner: vendor mgmt
Do browse-only users notice?	degraded browse is fine for R1’s existing customers	NON-RISK NR-2 — valid while scope holds

THE RULE One scenario, honestly walked: 1 sensitivity, 1 trade-off, 1 risk, 1 non-risk. A team that finds nothing isn’t looking.

Four moves under fire

1 · Steelman it

“So your concern is: a vendor we don’t control sits inside our revenue path — worst day, worst hour.” Restate it STRONGER than they said it.

2 · Answer to the scenario

Not taste, not fashion — trace the answer through a scenario and a diagram, and land on a measure.

3 · Concede honestly

“Correct — browse can be 60 s stale. We accepted that, it’s labeled, and here’s why.” Named costs read as competence.

4 · Park it, with a name

“Vendor SLA terms need procurement — logged, owner named, answer before charter.” Parking without an owner is dodging.

THE SCEPTICAL EXEC: “AND WHEN YOUR FEED DIES, WE JUST... STOP SELLING?”

Weak: “That basically never happens, and we have retries...” — vague, defensive, and now the room smells it.

Strong: “Yes — for as long as the outage lasts, ordering suspends. Browse survives on labeled data. We refuse stale-price orders on purpose — it’s our availability scenario. The open item is vendor restore time: logged, owned, due before charter.”

THE RULE You are not the defendant — the design is. Steelman, answer to the scenario, concede the real costs, park with owners.

Review scorecard — six dimensions, two comments

1 • Driver clarity	Could you repeat what winning means? Did the scenarios carry numbers?	/5
2 • Boundary reasoning	Boundaries argued from language and data ownership — not fashion?	/5
3 • Diagram honesty	Labeled arrows, own stores, externals dashed, marks where it hurts?	/5
4 • Trade-off candor	Costs named and accepted out loud — or hidden under adjectives?	/5
5 • Challenge conduct	Steelman → answer to the scenario → concede → park with an owner?	/5
6 • Honest tail	Open risks owned and dated, hand-offs routed, the bill acknowledged?	/5

One strength — be specific, quote them if you can

One thing to sharpen — one concrete change

THE RULE Score the six, then write the two comments — the comments are what the other team actually uses.

Put your design on trial

Same teams · inputs: the utility tree, container view, sequence diagram and ADR-007 worked through Chapters 3–5 — plus anything your team drafted alongside them · output: a findings wall in four buckets

1

Walk scenario one — your availability QAS

Finger on the diagrams, out loud, step by step. At every step: answered (cite the mechanism) or finding (write it down). No silent nodding.

2

Walk scenario two — your performance QAS

Same discipline. Where exactly is p95 measured? What melts first at market open — and what did you just discover you don't know?

3

File every finding, properly formed

Sensitivity & trade-off points: name the dial and what it pulls. Risks: BECAUSE condition → consequence, + owner. Non-risks: + the condition that would reopen them.

Minimum honest yield: 2 sensitivity/trade-off points · 2 risks with owners · 1 non-risk with its expiry. Found nothing? Walk slower — the worked example found four in one scenario.

The challenge drill — pairs, in role

Pair up with another team · challengers pick from the script · defenders answer with the four moves · then swap roles

SOX COMMITTEE CHAIR

“Show me — on your diagrams — the record trail for a purchase that FAILED halfway. Who wrote what, and when?”

SOX COMMITTEE CHAIR

“Your audit archive: who can delete from it? Prove it to me.”

TECHNOLOGY BOARD CTO

“You chose a managed event bus. When the Board standardizes on something else in two years, what moves — and what breaks?”

TECHNOLOGY BOARD CTO

“Why five services and one team? Defend the operational bill, line by line.”

BROKERAGE HEAD

“A customer bought at a 60-second-old price and lost money. Read me the complaint response.”

BROKERAGE HEAD

“Market opens, your search melts, customers walk. Which number on which dashboard catches it first?”

Every answer: steelman → scenario → concede → park. Skipping the steelman restarts the answer.

What strong answers had in common



They started from the challenge, not the defense

“So your concern is...” — the steelman bought three seconds of thought and the room’s respect.



They pointed at artifacts, not adjectives

Finger on the sequence diagram’s ! mark, the ADR’s consequences line, the QAS measure. Evidence has coordinates.



They conceded something real

“Yes — 60 seconds stale, labeled, accepted.” The conceded cost was the most credible sentence in the answer.



They parked with names and dates

“R-1, vendor management, before charter.” Parked ≠ dodged when it has an owner.

THE RULE These three personas are the room you will actually stand in — the drill is the rehearsal.

Reviewed — now make it harder to kill

- ✓ Name sensitivity and trade-off points in review language
- ✓ Record risks (condition → consequence, owned) and non-risks (with expiry)
- ✓ Run an ATAM-style walkthrough across five lenses at once
- ✓ Defend under challenge: steelman, scenario, concede, park

NEXT · CHAPTER 7

The review told you where it hurts.

Now harden it: failure modes, blast radius, the risks you hand to specialists — and cost as a design constraint, not a surprise.

07

Hardening, Hand-offs, Cost & Effort

The review told you where it hurts. Hardening is what you do about it: trace failure through the dependency chain, contain the blast radius, hand the risks you can't close to the specialists who can — and treat cost and effort as design constraints with the same rank as performance.

THE COURSE

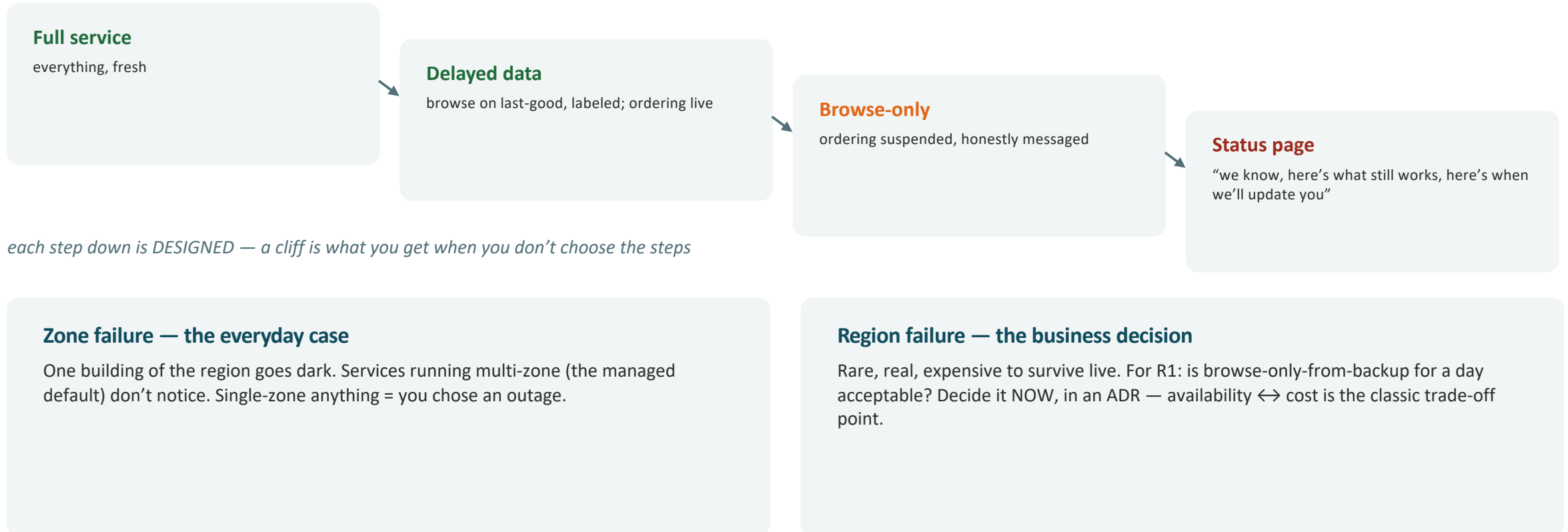
- 1 Mining the Backlog — Forces & ASRs
- 2 Use Case Modeling — Behavior & a First Candidate
- 3 Quality Attributes, Scenarios & the Utility Tree
- 4 Styles, DDD & Microservices — and the Cloud
- 5 Documenting the Solution
- 6 Analyzing & Reviewing — the ATAM Discipline
- 7 Hardening, Hand-offs, Cost & Effort**

Failure modes: ask “what dies when THIS dies?”

Third-party feed dies	browse degrades (labeled) · ordering suspends	designed — ADR-007 + availability scenario
Event bus dies	orders still complete · records & receipts LAG, then catch up	acceptable IF lag is bounded & monitored
Records store dies	does selling continue? UNANSWERED — the audit rule says nothing may go unrecorded	finding! decide: block orders or buffer safely
Identity provider dies	nobody signs in — EVERYTHING stops	single point of failure — nobody drew it
A cloud ZONE dies	one copy of each service gone	multi-zone services = non-event; single-zone = outage

THE RULE Kill each dependency on paper and follow the chain — the unanswered rows are your hardening backlog.

Blast radius — and the degradation ladder



THE RULE Design the downward steps, not just the happy path — a chosen degradation keeps trust a cliff destroys.

Six hardening moves that cost almost nothing — now



Timeouts on every sync arrow

no call waits forever — the checkout page must never hang on a feed



Idempotency keys on writes

a retried purchase must not double-buy — key on the order, dedupe on arrival



Breakers + a named fallback

fail fast toward the ladder step you CHOSE, not a white screen



Multi-zone by default

the checkbox that deletes a whole failure class — verify it's actually on



Buffer the must-never-lose

audit events land in durable storage FIRST; consumers can lag, loss cannot



Alert on your QAS measures

you already defined the thresholds — your scenario's 'alert $\leq$ 2 min' is a config line now

THE RULE Hardening is mostly cheap at design time and brutal at 2 a.m. — spend the cheap money first.

Unresolved risks & specialist hand-offs — the architect’s exit artifact

Open risk	What we did	What remains	Hand to	By
R-1 Supplier-feed SLA unconfirmed	breaker + ladder designed	contract terms, restore-time commitment	Vendor management	before charter
R-2 PII & threat surface unreviewed	authZ at gateway + between services drawn	full STRIDE pass, pen-test plan	Security engineering	before build starts
R-3 Region-failure posture undecided	options priced (ladder vs live-live)	business call: cost vs downtime tolerance	CTO + product owner	charter meeting
R-4 Audit archive legal retention	immutable storage + lock designed	retention years? deletion exceptions?	Legal / compliance	before launch

Zero open risks = dishonest, not done. Your job as the generalist is to find, frame, mitigate what you can — and route the rest to people licensed to close it, with a date. Panels trust the team whose open list looks like this.

THE RULE Hand off with the risk **FRAMED**: what we did, what remains, who owns it, by when — that row is the deliverable.

Cost & effort — the bill is a design property

Compute

instance-seconds & requests — chatty designs buy more of both

Storage

data + backups + the audit archive that only ever grows

Egress

data LEAVING (cross-region, to the internet) bills; entering doesn't

Per-request services

bus messages, gateway calls, function invocations — millions × tiny

WORKED ARITHMETIC — THE CHATTY PAGE (COUNTS ILLUSTRATIVE)

Product detail page → 4 sync calls (catalog, price, eligibility, history) × 50,000 views/day = 200,000 backend invocations/day

One composite endpoint with a short cache → 50,000/day. Same page, 4× fewer invocations — one arrow moved on one diagram.

And effort is a currency too: every service you add bills engineer-hours forever — pipelines, upgrades, on-call. A design your team can't afford to RUN is a failed design with excellent diagrams. Your cost scenario is the meter; read it monthly.

THE RULE Ask “what does this arrow cost per month?” in the same breath as “is it fast?” — cost is a quality attribute, not a postmortem.

Break it on paper, then bill it

Same teams · inputs: the container view and sequence diagram worked in Chapter 5, plus your Lab 2 findings wall · output: one hardening pass + one hand-off table + one cost check

1

Kill your scariest dependency

Pick it (feed? bus? identity?). Walk the chain out loud: what degrades, what stops, what loses data? Draw your degradation ladder — name each step.

2

Apply two hardening moves

From the six: the two that close YOUR chain's worst gaps. Write each as a config-style line (timeout X, key on Y, buffer Z first).

3

Write your hand-off table

2–3 open risks in the five-column form: risk → what we did → what remains → hand to → by when. Real names of roles, real dates.

4

Run the 5-line cost check

Your chattiest page or flow: calls × daily volume = invocations/day. Name the bill driver it feeds. Find ONE change that cuts it — and say what that change trades away.

Model answers — and what's now in your hands

Model answers (feed as scariest)

- Ladder: full → delayed (labeled) → browse-only → status page; each step messaged, none a surprise
- Moves: breaker w/ fallback=ladder-step-2 · audit events buffered to durable storage before consumers
- Hand-off row: “R-2 threat surface — authZ drawn; STRIDE pass remains — Security engineering — before build”
- Cost: 4 calls × 50,000 views = 200,000/day → composite+cache = 50,000 (trade: staleness ≤ 30 s on the page)

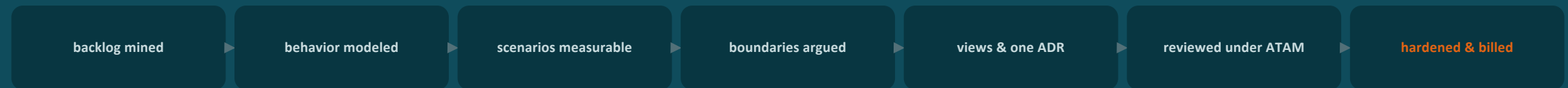
YOUR DESIGN PACK — COMPLETE

- Utility tree with (H,H) leaves
- Context map + container view + sequence
- Deployment notes + one ADR
- Findings: sensitivities, trade-offs, risks, non-risks
- Degradation ladder + hand-off table + cost check

This is the pack you would carry into a real architecture review.

SESSION 12 · COMPLETE

You arrived with a backlog. You leave with a design you can defend.



What you take to the job: the loop (design → analyze → refine), the vocabulary (scenarios, trade-offs, risks, non-risks), the artifacts (tree, views, ADRs, hand-offs) — and the nerve to be challenged without flinching.

Next in the programme: you build it. On cloud. Bring this design.